Project 4 Report

ME 249

Parker Yang, Md Elius, Van P. Carey
2025. 12. 18

The following report contains project 4 for ME 249. The following work was done between Parker Yang and Md Elius. The coding work involved in the project was done in collaboration. Both of the teammates participated in data preparation and modifications as well as the trainning. The writing of the report was divided among us. Each took portions in writing the report within the two different parts of the report, dividing them by each Tasks.

Part 1

Task 4.1.1

The following figures show the trained neural network model constructed with the given training data (T_{air}, I_D, R_L) to achieve the output parameters (V_L,W). Our model was able to achieve the smallest loss value of 0.024 with the dataset split between training (36 samples) and validation (18 samples).

The attached figure below shows the predicted vs actual for the training data and validation data with a y=x reference line to assess the agreement.

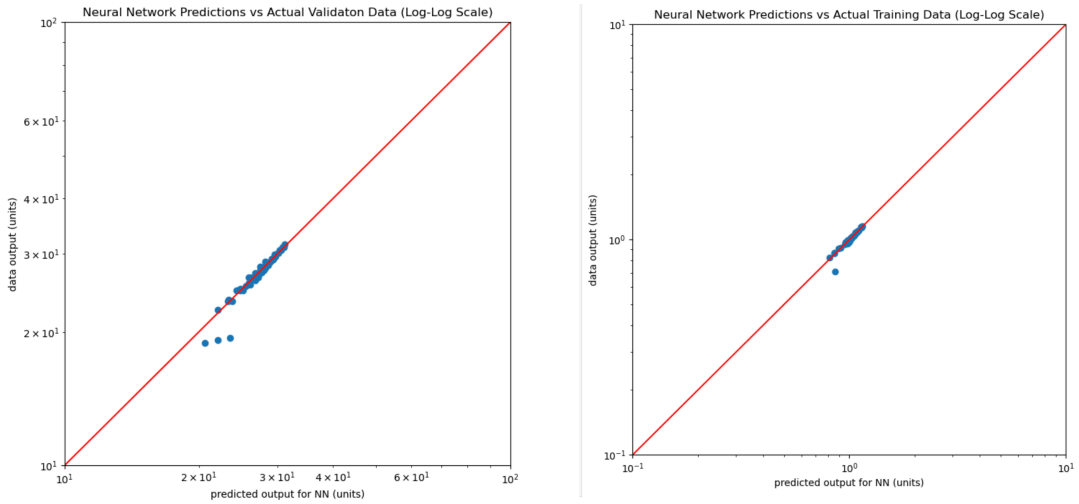


Figure 1. Log-log plots of model vs. actual and training data (left,right)

The figures below also showed the point by point comparison of the power output.

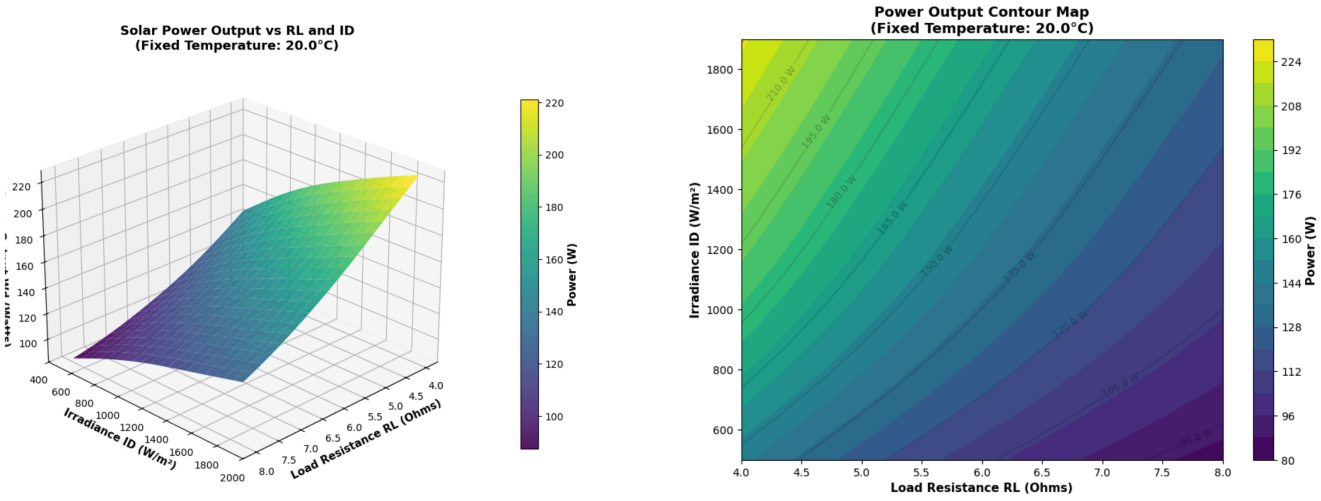


Figure 2. Results from the model showing the power output

Task 4.1.2

When comparing the two models with different numbers of hidden layers, the first model with 3 hidden layers over 4 showed better performance. Overall, both models showed strong agreement with the data. However, the 3 hidden layer model achieved a better minimum training loss of 0.0239 while the 4 hidden layer model achieved a minimum better training loss of 0.0325. The 4 hidden layer model struggled to achieve the goal minimum training loss below 0.025.

In terms of looking for possible overfitting, there were not any clear signs of overfitting. Both of the models seem to achieve the plateau with early stoppage.

The two models with different numbers of hidden layers showed how additional hidden layers do not automatically improve the model performance. With a limited dataset, additional layer structure brought challenges in optimization without any significant benefits. Both of the models seem to cluster around the y=x reference line, showing reasonable performance of the models. The surface plots and contour maps both displayed a reasonable physical trend with increase in power output with irradiance and decrease with load resistance.

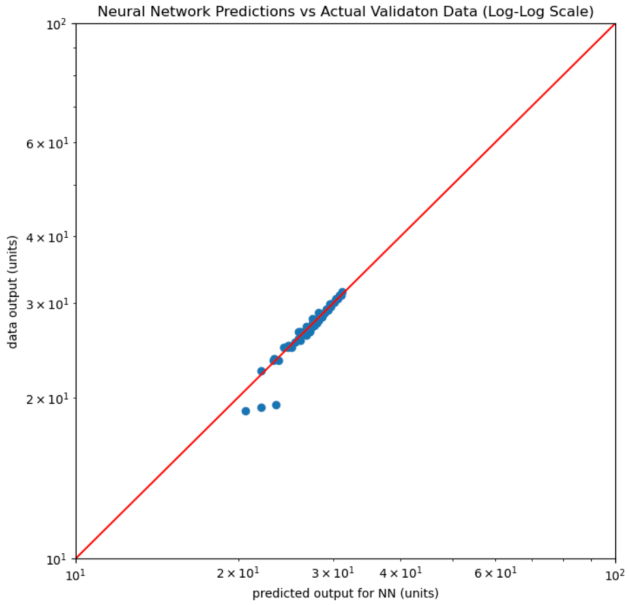


Figure 3. Result from the model with 3 hidden layers

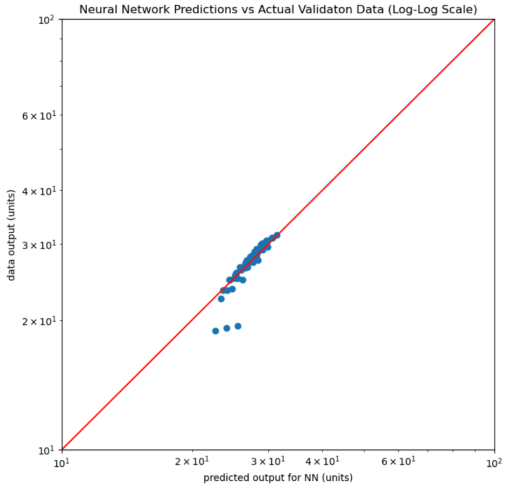


Figure 4. Result from the model with 4 hidden layers

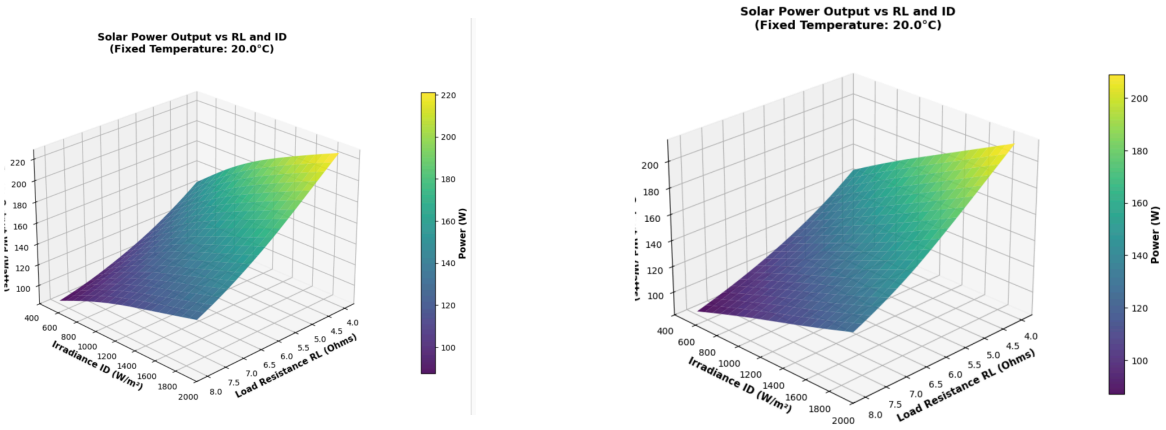


Figure 5. Results from the model with 3 and 4 hidden layers (left,right)

Part 2

Task 4.2.1

The following model was developed to evaluate a machine-learning based model that can predict which mode will produce the most power for a specified set of operating conditions. Specifically, the objective is to use the model that predicts the mode number “M” max for maximum performance and output power of the solar PV panel for that mode, at a given set of operating conditions for model-based control of the PV system.

For this model, four different inputs (M, T_{air}, I_D,R_L) were chosen for assessing two output parameters (V_L,W). The model used 4 hidden layers having 64, 62, 62, and 48 neurons to successfully achieve smallest loss = 0.0199.

The figure below shows a separate log-log plots of predicted power output vs. data value power output, for each set of data point operating conditions with a mean absolute error of 80.3074 Watts.

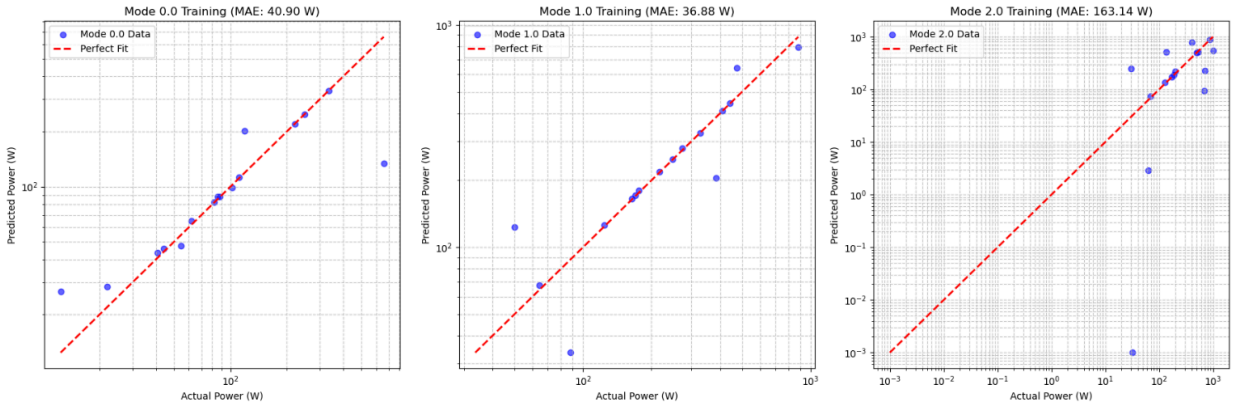


Figure 6. Log-log plots of model validation across three different modes (left, middle, and right)

The following figures shows comparison of the model predictions to the normalized validation data with a mean absolute error of 228.60 Watts.

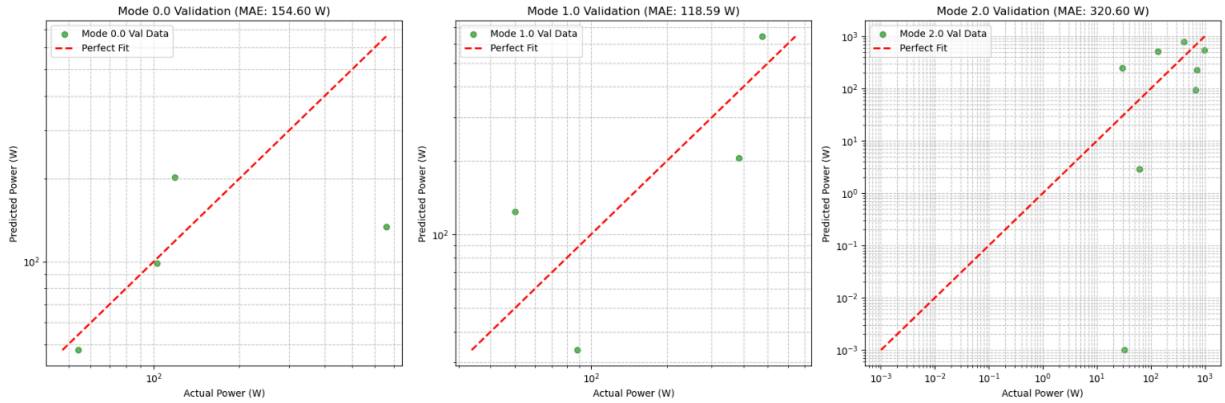


Figure 7. Model validation across three different modes with normalized dataset

Task 4.2.2

(h)

The trained model achieved a loss below the target threshold of 0.020 on the training data.

Dataset	Loss	Accuracy
Training	0.0076	1.0
Validation	0.7848	0.80

(i) The training loss is over 100× smaller than the validation loss (0.00765 vs. 0.78485), and accuracy drops from 100% to 80% on the validation data. This large gap indicates that the model has memorized the training data but does not generalize as well to unseen data, which is characteristic of overfitting.

(j) Summary with Dropout (p = 0.25)

Dataset	Loss	Accuracy
Training	0.00185	1.0
Validation	0.89129	0.80

There is no evidence that dropout reduced overfitting in this case. Quantitatively, the gap between training and validation loss increased compared to the no-dropout model:

- Training loss decreased from 0.00765 to 0.00185
- Validation loss increased from 0.78485 to 0.89129
- Validation accuracy remained unchanged at 80%

The model still achieves perfect training accuracy while validation performance does not improve, indicating that overfitting persists and dropout did not make a meaningful difference for this dataset and model size.

(k)

T_air (deg C)	I_d (W/m^2)	R_l (ohms)	M_max, int Pred by Task 2.2 model	W predicted by Task 2.1 model for M =0	W predicted by task 2.1 model for M =1	W Predicted by task 2.1 model for M =2
10.0	200	50	1	41.7	170.95	62.54
20.0	200	130	2	10.99	63.40	168.56
10.0	500	40	1	55.76	230.15	156.83
20.0	500	80	2	86.59	170.05	258.15
20.0	700	30	2	81.99	332.18	503.86
20.0	700	55	2	52.83	190.36	361.60
10.0	1000	12	1	213.81	871.57	831.02
20.0	1000	25	2	103.46	435.25	881.90
20.0	1000	39	2	67.27	251.11	574.53

Based on the Part 2 results, the Task 4.2.2 neural network model with and without dropout can accurately control the switch setting for the multi-mode 4-panel PV system in Fig. 2. For all tested operating conditions, the predicted switch mode MMM corresponds to the configuration that yields the highest or near-highest predicted power from the Task 2.1 model.

Quantitatively, in every case shown, the mode selected by the classifier aligns with the maximum predicted power among the three configurations, with only small power differences when alternatives are close. Despite evidence of overfitting in validation loss, the mode-selection accuracy is sufficient for effective control, indicating robust practical performance.

Appendix

In [4]:
```
'''>>>>> start CodeP4.2F25
    V.P. Carey ME249, Fall 2025

Intro to Neural Network Modeling
Keras model for PV power system'''

#import useful packages
import keras
import pandas as pd
from keras.models import Sequential
import numpy as np
import keras.backend as kb
import tensorflow as tf
#the follwoing 2 lines are only needed for Mac OS machines
import os
os.environ['KMP_DUPLICATE_LIB_OK']='True'


# ===== PART 1: Define raw data first, convert temp to Kelvin, then calculat
# Raw input data: Air temp (degC), ID (W/sqm), load resistance (ohms)
xdata_celcius = [[-10.0, 1550, 4.464],
  [-10.0, 1850, 4.464],
  [10.0, 1550, 4.464],
  [10.0, 1850, 4.464],
  [30.0, 1550, 4.464],
  [30.0, 1850, 4.464],
  [-10.0, 1550, 6.696],
  [-10.0, 1850, 6.696],
  [10.0, 1550, 6.695],
  [10.0, 1850, 6.696],
  [30.0, 1550, 6.696],
  [30.0, 1850, 6.698],
  [-10.0, 1550, 8.928],
  [-10.0, 1850, 8.928],
  [10.0, 1550, 8.928],
  [10.0, 1850, 8.928],
  [30.0, 1550, 8.928],
  [30.0, 1850, 8.928],

### x Data from Code 3.1
  [-10.0, 350, 4.464],
  [-10.0, 650, 4.464],
  [-10.0, 950, 4.464],
  [-10.0, 1250, 4.464],
  [10.0, 350, 4.464],
  [10.0, 650, 4.464],
  [10.0, 950, 4.464],
  [10.0, 1250, 4.464],
  [30.0, 350, 4.464],
  [30.0, 650, 4.464],
  [30.0, 950, 4.464],
  [30.0, 1250, 4.466],
  [-10.0, 350, 6.696],
  [-10.0, 650, 6.696],
```

```
  [-10.0, 950, 6.696],
  [-10.0, 1250, 6.696],
  [10.0, 350, 6.696],
  [10.0, 650, 6.696],
  [10.0, 950, 6.696],
  [10.0, 1250, 6.696],
  [30.0, 350, 6.696],
  [30.0, 650, 6.696],
  [30.0, 950, 6.696],
  [30.0, 1250, 6.696],
  [-10.0, 350, 8.928],
  [-10.0, 650, 8.928],
  [-10.0, 950, 8.928],
  [-10.0, 1250, 8.928],
  [10.0, 350, 8.928],
  [10.0, 650, 8.928],
  [10.0, 950, 8.928],
  [10.0, 1250, 8.928],
  [30.0, 350, 8.928],
  [30.0, 650, 8.928],
  [30.0, 950, 8.928],
  [30.0, 1250, 8.928]]


#Part 1 output HI FLUX DATA: load voltage (V) and Power out (W)
ydata_raw = [[26.1, 152.8],
 [26.5, 172.7],
 [27.9, 191.4],
 [28.3, 198.0],
 [29.6, 217.8],
 [30.1, 224.4],
 [26.9, 118.8],
 [27.2, 122.1],
 [28.8, 136.4],
 [29.1, 139.6],
 [30.6, 154.0],
 [31.0, 158.4],
 [27.3, 92.4],
 [27.6, 94.6],
 [29.2, 105.6],
 [29.6, 107.8],
 [31.1, 119.9],
 [31.5, 123.2],
### y Data from 3.1
 [18.9, 80.3],
 [23.5, 124.6],
 [24.8, 138.6],
 [25.6, 146.9],
 [19.2, 83.1],
 [25.0, 140.5],
 [26.5, 157.6],
 [27.3, 167.5],
 [19.4, 84.7],
 [26.4, 156.7],
 [28.1, 177.7],
 [29.0, 189.4],
```

```
 [22.4, 75.2],
 [24.8, 92.2],
 [25.8, 99.7],
 [26.4, 104.6],
 [23.6, 83.7],
 [26.5, 104.9],
 [27.6, 113.8],
 [28.3, 119.6],
 [24.8, 92.0],
 [28.1, 118.2],
 [29.3, 128.8],
 [30.1, 135.5],
 [23.5, 62.2],
 [25.4, 72.5],
 [26.3, 77.6],
 [26.9, 81.1],
 [25.0, 70.3],
 [27.1, 82.7],
 [28.1, 88.7],
 [28.7, 92.8],
 [26.5, 78.6],
 [28.8, 93.5],
 [29.9, 100.5],
 [30.6, 105.2]]


# Convert temperature from Celsius to Kelvin (K = degC + 273.15)
xdata_kelvin= [[row[0] + 273.15, row[1], row[2]] for row in xdata_celcius]

print("===== Temperature Conversion =====")
print("First 3 samples (Celsius -> Kelvin):")
for i in range(3):
    print(f"  {xdata_celcius[i][0]:6.2f} °C  ->  {xdata_kelvin[i][0]:6.2f} K
print()


# Convert to numpy arrays and calculate medians
xarray = np.array(xdata_kelvin)
yarray = np.array(ydata_raw)

# Calculate input variable medians from data
Tamed = np.median(xarray[:, 0])   # Air temperature median (K)
IDmed = np.median(xarray[:, 1])   # Irradiance (ID) median (W/sqm)
RLmed = np.median(xarray[:, 2])   # Load resistance median (ohms)

# Calculate output variable medians from data
VLmed = np.median(yarray[:, 0])   # Load voltage (VL) median (V)
Wdmed = np.median(yarray[:, 1])   # Power output (Wd) median (W)

print("===== Calculated Median Values =====")
print(f"Temperature median: {Tamed:.2f} K")
print(f"Irradiance median: {IDmed:.2f} W/sqm")
print(f"Load resistance median: {RLmed:.4f} ohms")
print(f"Voltage median: {VLmed:.2f} V")
print(f"Power median: {Wdmed:.2f} W\n")
```

```
# ===== PART 2: Create normalized input data array =====
# Normalize input data by dividing by medians
xdata = []
for row in xdata_kelvin:
    xdata.append([row[0]/Tamed, row[1]/IDmed, row[2]/RLmed])

xarray = np.array(xdata)
print("===== Normalized Input Data =====")
print("First 5 rows (normalized):")
print(xarray[:5])
print(f"Input shape: {xarray.shape}\n")

# ===== PART 3: Create normalized output data array =====
# Normalize output data by dividing by medians
ydata = []
for row in ydata_raw:
    ydata.append([row[0]/VLmed, row[1]/Wdmed])

yarray = np.array(ydata)
print("===== Normalized Output Data =====")
print("First 5 rows (normalized):")
print(yarray[:5])
print(f"Output shape: {yarray.shape}\n")

# ===== PART 4: Split data into training (2/3) and validation (1/3) sets ===
# Set random seed for reproducibility
np.random.seed(42)

# Generate random indices and shuffle them
total_samples = len(xarray)
indices = np.arange(total_samples)
np.random.shuffle(indices)

# Calculate split point (2/3 for training, 1/3 for validation)
split_point = int(2/3 * total_samples)

# Split indices
train_indices = indices[:split_point]
val_indices = indices[split_point:]

# Create training and validation sets
x_train = xarray[train_indices]
y_train = yarray[train_indices]
x_val = xarray[val_indices]
y_val = yarray[val_indices]

print("===== Data Split =====")
print(f"Total samples: {total_samples}")
print(f"Training set size: {len(x_train)} ({len(x_train)/total_samples*100:.
print(f"Validation set size: {len(x_val)} ({len(x_val)/total_samples*100:.1f
print(f"\nTraining input shape: {x_train.shape}")
print(f"Training output shape: {y_train.shape}")
print(f"Validation input shape: {x_val.shape}")
print(f"Validation output shape: {y_val.shape}\n")
```

```
===== Temperature Conversion =====
First 3 samples (Celsius -> Kelvin):
  -10.00 °C  ->  263.15 K
  -10.00 °C  ->  263.15 K
   10.00 °C  ->  283.15 K

===== Calculated Median Values =====
Temperature median: 283.15 K
Irradiance median: 1100.00 W/sqm
Load resistance median: 6.6960 ohms
Voltage median: 27.25 V
Power median: 116.00 W

===== Normalized Input Data =====
First 5 rows (normalized):
[[0.92936606 1.40909091 0.66666667]
 [0.92936606 1.68181818 0.66666667]
 [1.         1.40909091 0.66666667]
 [1.         1.68181818 0.66666667]
 [1.07063394 1.40909091 0.66666667]]
Input shape: (54, 3)

===== Normalized Output Data =====
First 5 rows (normalized):
[[0.95779817 1.31724138]
 [0.97247706 1.4887931 ]
 [1.02385321 1.65      ]
 [1.03853211 1.70689655]
 [1.08623853 1.87758621]]
Output shape: (54, 2)

===== Data Split =====
Total samples: 54
Training set size: 36 (66.7%)
Validation set size: 18 (33.3%)

Training input shape: (36, 3)
Training output shape: (36, 2)
Validation input shape: (18, 3)
Validation output shape: (18, 2)
```

In [25]:

```
# define neural network model

#As seen below, we have created four dense layers.
#A dense layer is a layer in neural network that's fully connected.
#In other words, all the neurons in one layer are connected to all other neu
#In the first layer, we need to provide the input shape, which is 1 in our c
#The activation function we have chosen is elu, which stands for exponential

from keras import backend as K
#initialize weights with values between -0.2 and 0.5
# initializer = keras.initializers.RandomUniform(minval= -0.2, maxval=0.5)
initializer = keras.initializers.RandomUniform(minval= -0.2, maxval=0.5)

model = keras.Sequential([
```

```
    keras.layers.Dense(6, activation='elu', input_shape=[3],  kernel_initial
    keras.layers.Dense(8, activation='elu',  kernel_initializer=initializer)
    keras.layers.Dense(14, activation='elu', kernel_initializer=initializer)
    keras.layers.Dense(8, activation='elu',  kernel_initializer=initializer)
    keras.layers.Dense(2,  kernel_initializer=initializer)
  ])
```

In [43]:
```
#We're using RMSprop as our optimizer here. RMSprop stands for Root Mean Squ
#It's one of the most popular gradient descent optimization algorithms for d
#RMSprop is an optimizer that's reliable and fast.
#We're compiling the mode using the model.compile function. The loss functio
#is mean squared error. After the compilation of the model, we'll use the fi
#Number of epochs can be varied.

#from tf.keras import optimizers
# rms = keras.optimizers.RMSprop(0.020)
rms = keras.optimizers.RMSprop(0.003)
model.compile(loss='mean_absolute_error',optimizer=rms)
```

In [68]:
```
#After the compilation of the model, we'll use the fit method with 500 epoch
#I started with epochs value of 100 and then tested the model after training
#The prediction was not that good. Then I modified the number of epochs to 2
#Accuracy had improved slightly, but figured I'd give it one more try. Final
#I found acceptable prediction accuracy.

#The fit method takes three parameters; namely, x, y, and number of epochs.
#During model training, if all the batches of data are seen by the model onc
#we say that one epoch has been completed.

# Add an early stopping callback
es = keras.callbacks.EarlyStopping(
    monitor='loss',
    mode='min',
    patience = 80,
    restore_best_weights = True,
    verbose=1)
# Add a checkpoint where loss is minimum, and save that model
mc = keras.callbacks.ModelCheckpoint('best_model.keras', monitor='loss',
                     mode='min',  verbose=1, save_best_only=True)

historyData = model.fit(x_train,y_train,epochs=800,callbacks=[es])

loss_hist = historyData.history['loss']
#The above line will return a dictionary, access it's info like this:
best_epoch = np.argmin(historyData.history['loss']) + 1
print ('best epoch = ', best_epoch)
print('smallest loss =', np.min(loss_hist))

model.save('./best_model.keras')
```

```
Epoch 1/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0409
Epoch 2/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0458
Epoch 3/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0383
Epoch 4/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0365
Epoch 5/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0315
Epoch 6/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0380
Epoch 7/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0313
Epoch 8/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0349
Epoch 9/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0395
Epoch 10/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0305
Epoch 11/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0449
Epoch 12/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0272
Epoch 13/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0301
Epoch 14/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0339
Epoch 15/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0636
Epoch 16/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0670
Epoch 17/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0273
Epoch 18/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0354
Epoch 19/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0313
Epoch 20/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0339
Epoch 21/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0340
Epoch 22/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0457
Epoch 23/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0291
Epoch 24/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0523
Epoch 25/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0420
Epoch 26/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0291
Epoch 27/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0355
Epoch 28/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0432
```

```
Epoch 29/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0377
Epoch 30/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0374
Epoch 31/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0364
Epoch 32/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0429
Epoch 33/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0278
Epoch 34/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0263
Epoch 35/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0347
Epoch 36/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0462
Epoch 37/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0350
Epoch 38/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0528
Epoch 39/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0279
Epoch 40/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0348
Epoch 41/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0304
Epoch 42/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0281
Epoch 43/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0580
Epoch 44/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0512
Epoch 45/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0325
Epoch 46/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0452
Epoch 47/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0329
Epoch 48/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0545
Epoch 49/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0306
Epoch 50/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0433
Epoch 51/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0291
Epoch 52/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0370
Epoch 53/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0268
Epoch 54/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0590
Epoch 55/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0307
Epoch 56/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0325
```

```
Epoch 57/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0395
Epoch 58/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0276
Epoch 59/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0410
Epoch 60/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0419
Epoch 61/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0383
Epoch 62/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0400
Epoch 63/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0401
Epoch 64/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0313
Epoch 65/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0296
Epoch 66/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0469
Epoch 67/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0449
Epoch 68/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0290
Epoch 69/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0287
Epoch 70/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0285
Epoch 71/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0310
Epoch 72/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0519
Epoch 73/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0289
Epoch 74/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0325
Epoch 75/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0429
Epoch 76/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0453
Epoch 77/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0333
Epoch 78/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0390
Epoch 79/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0318
Epoch 80/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0373
Epoch 81/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0615
Epoch 82/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0311
Epoch 83/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0330
Epoch 84/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0267
```

```
Epoch 85/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0533
Epoch 86/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0294
Epoch 87/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0595
Epoch 88/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0323
Epoch 89/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0413
Epoch 90/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0429
Epoch 91/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0428
Epoch 92/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0280
Epoch 93/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0454
Epoch 94/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0303
Epoch 95/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0259
Epoch 96/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0369
Epoch 97/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0286
Epoch 98/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0568
Epoch 99/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0286
Epoch 100/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0740
Epoch 101/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0333
Epoch 102/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0253
Epoch 103/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0449
Epoch 104/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0374
Epoch 105/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0399
Epoch 106/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0382
Epoch 107/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0381
Epoch 108/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0378
Epoch 109/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0262
Epoch 110/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0393
Epoch 111/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0396
Epoch 112/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0365
```

```
Epoch 113/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0276
Epoch 114/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0484
Epoch 115/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0315
Epoch 116/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0444
Epoch 117/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0283
Epoch 118/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0278
Epoch 119/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0337
Epoch 120/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0501
Epoch 121/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0414
Epoch 122/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0454
Epoch 123/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0275
Epoch 124/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0328
Epoch 125/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0279
Epoch 126/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0338
Epoch 127/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0345
Epoch 128/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0283
Epoch 129/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0583
Epoch 130/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0300
Epoch 131/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0580
Epoch 132/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0311
Epoch 133/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0535
Epoch 134/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0428
Epoch 135/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0399
Epoch 136/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0387
Epoch 137/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0317
Epoch 138/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0379
Epoch 139/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0296
Epoch 140/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0387
```

```
Epoch 141/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0306
Epoch 142/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0529
Epoch 143/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0309
Epoch 144/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0435
Epoch 145/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0253
Epoch 146/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0366
Epoch 147/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0270
Epoch 148/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0333
Epoch 149/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0425
Epoch 150/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0299
Epoch 151/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0403
Epoch 152/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0287
Epoch 153/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0562
Epoch 154/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0307
Epoch 155/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0483
Epoch 156/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0334
Epoch 157/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0264
Epoch 158/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0268
Epoch 159/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0289
Epoch 160/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0474
Epoch 161/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0417
Epoch 162/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0305
Epoch 163/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0448
Epoch 164/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0491
Epoch 165/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0353
Epoch 166/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0422
Epoch 167/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0471
Epoch 168/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0288
```

```
Epoch 169/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0255
Epoch 170/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0505
Epoch 171/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0304
Epoch 172/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0340
Epoch 173/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0349
Epoch 174/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0571
Epoch 175/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0368
Epoch 176/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0315
Epoch 177/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0442
Epoch 178/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0420
Epoch 179/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0262
Epoch 180/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0262
Epoch 181/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0434
Epoch 182/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0404
Epoch 183/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0476
Epoch 184/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0465
Epoch 185/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0276
Epoch 186/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0434
Epoch 187/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0431
Epoch 188/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0346
Epoch 189/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0353
Epoch 190/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0473
Epoch 191/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0448
Epoch 192/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0259
Epoch 193/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0451
Epoch 194/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0253
Epoch 195/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0334
Epoch 196/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0421
```

```
Epoch 197/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0336
Epoch 198/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0420
Epoch 199/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0371
Epoch 200/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0432
Epoch 201/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0392
Epoch 202/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0356
Epoch 203/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0283
Epoch 204/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0566
Epoch 205/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0334
Epoch 206/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0339
Epoch 207/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0385
Epoch 208/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0459
Epoch 209/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0322
Epoch 210/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0314
Epoch 211/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0330
Epoch 212/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0460
Epoch 213/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0391
Epoch 214/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0268
Epoch 215/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0458
Epoch 216/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0255
Epoch 217/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0360
Epoch 218/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0350
Epoch 219/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0293
Epoch 220/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0286
Epoch 221/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0421
Epoch 222/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0336
Epoch 223/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0339
Epoch 224/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0350
```

```
Epoch 225/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0573
Epoch 226/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0291
Epoch 227/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0308
Epoch 228/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0286
Epoch 229/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0434
Epoch 230/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0371
Epoch 231/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0342
Epoch 232/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0348
Epoch 233/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0466
Epoch 234/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0297
Epoch 235/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0290
Epoch 236/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0304
Epoch 237/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0392
Epoch 238/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0381
Epoch 239/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0289
Epoch 240/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0259
Epoch 241/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0499
Epoch 242/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0451
Epoch 243/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0446
Epoch 244/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0262
Epoch 245/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0309
Epoch 246/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0305
Epoch 247/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0309
Epoch 248/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0293
Epoch 249/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0402
Epoch 250/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0399
Epoch 251/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0298
Epoch 252/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0423
```

```
Epoch 253/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0431
Epoch 254/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0325
Epoch 255/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0292
Epoch 256/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0516
Epoch 257/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0263
Epoch 258/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0332
Epoch 259/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0676
Epoch 260/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0257
Epoch 261/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0342
Epoch 262/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0426
Epoch 263/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0252
Epoch 264/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0476
Epoch 265/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0353
Epoch 266/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0491
Epoch 267/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0267
Epoch 268/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0330
Epoch 269/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0466
Epoch 270/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0368
Epoch 271/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0256
Epoch 272/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0353
Epoch 273/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0409
Epoch 274/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0272
Epoch 275/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0406
Epoch 276/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0281
Epoch 277/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0425
Epoch 278/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0449
Epoch 279/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0314
Epoch 280/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0280
```

```
Epoch 281/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0469
Epoch 282/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0464
Epoch 283/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0307
Epoch 284/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0315
Epoch 285/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0405
Epoch 286/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.0423
Epoch 287/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0461
Epoch 288/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0324
Epoch 289/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0239
Epoch 290/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0254
Epoch 291/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0563
Epoch 292/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0318
Epoch 293/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0303
Epoch 294/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0441
Epoch 295/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0254
Epoch 296/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0322
Epoch 297/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0447
Epoch 298/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0283
Epoch 299/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0270
Epoch 300/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0506
Epoch 301/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0286
Epoch 302/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0293
Epoch 303/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0467
Epoch 304/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0436
Epoch 305/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0331
Epoch 306/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0458
Epoch 307/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0451
Epoch 308/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0326
```

```
Epoch 309/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0358
Epoch 310/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0415
Epoch 311/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0310
Epoch 312/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0389
Epoch 313/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0521
Epoch 314/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0494
Epoch 315/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0301
Epoch 316/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0289
Epoch 317/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0345
Epoch 318/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0339
Epoch 319/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0309
Epoch 320/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0293
Epoch 321/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0435
Epoch 322/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0410
Epoch 323/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0436
Epoch 324/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0273
Epoch 325/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0291
Epoch 326/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0316
Epoch 327/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0302
Epoch 328/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0355
Epoch 329/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0353
Epoch 330/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0357
Epoch 331/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0298
Epoch 332/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0377
Epoch 333/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0334
Epoch 334/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0375
Epoch 335/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0343
Epoch 336/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0285
```

```
Epoch 337/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0297
Epoch 338/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0466
Epoch 339/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0405
Epoch 340/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0352
Epoch 341/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0498
Epoch 342/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0258
Epoch 343/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0502
Epoch 344/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0321
Epoch 345/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0372
Epoch 346/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0282
Epoch 347/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0326
Epoch 348/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0547
Epoch 349/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0320
Epoch 350/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0350
Epoch 351/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0260
Epoch 352/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0553
Epoch 353/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0515
Epoch 354/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0337
Epoch 355/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0261
Epoch 356/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0301
Epoch 357/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0271
Epoch 358/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0309
Epoch 359/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0381
Epoch 360/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0305
Epoch 361/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0402
Epoch 362/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0433
Epoch 363/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0476
Epoch 364/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0440
```

```
Epoch 365/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0345
Epoch 366/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0280
Epoch 367/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0483
Epoch 368/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0351
Epoch 369/800
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.0334
Epoch 369: early stopping
Restoring model weights from the end of the best epoch: 289.
best epoch =  289
smallest loss = 0.023882657289505005
```

In [69]:
```
# This line of code can be used to reconstruct the saved model.

recon_model = keras.models.load_model("best_model.keras")

# the name of the model is now "recon_model". You can then use this model to
# See the previous project for code to do the comparisons.
```

In [1]:
```
test = []
outpt=[]
predictions = model.predict(x_val)

#put in a loop to print comparison for all data points
for i in range(len(xarray)):
    test = [[xarray[i][0], xarray[i][1], xarray[i][2]]]
    testarray = np.array(test)
    outpt = model.predict(testarray, verbose=0)
    print('row [' + str(i) + '] data:  T1= ', xarray[i][0]*Tamed, ', ID= ',
        ', RL= ', xarray[i][2]*RLmed, ', power= ', yarray[i][1]*Wdmed, \
        ',  predicted power = ', outpt[0][1]*Wdmed)

#SETTING UP PLOT
%matplotlib inline
# importing the required module
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [8, 8] # for square canvas
#========
'''CALCULATE PREDICTED VALUES AND RETRIEVE DATA VALUES TO PLOT'''
# Denormalize for plotting
predictions_denorm = predictions[:, 0] * VLmed
actual_denorm = yarray[:, 0] * VLmed

plt.scatter(predictions_denorm, actual_denorm)
plt.title('Neural Network Predictions vs Actual Validaton Data (Log-Log Scal
plt.xlabel('predicted output for NN (units)')
plt.ylabel('data output (units)')
plt.loglog()
plt.xlim(xmax=100, xmin=10)
plt.ylim(ymax=100, ymin=10)
# Generate red y=x line using logspace for proper log scale
x_data = np.logspace(np.log10(1.0), np.log10(100.0), num=100)
y_data = x_data
```

```
plt.plot(x_data, y_data, color='red')
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[1], line 3
      1 test = []
      2 outpt=[]
----> 3 predictions = model.predict(x_val)
      5 #put in a loop to print comparison for all data points
      6 for i in range(len(xarray)):

NameError: name 'model' is not defined
```

In [71]:
```
test = []
outpt=[]
predictions = model.predict(x_train)

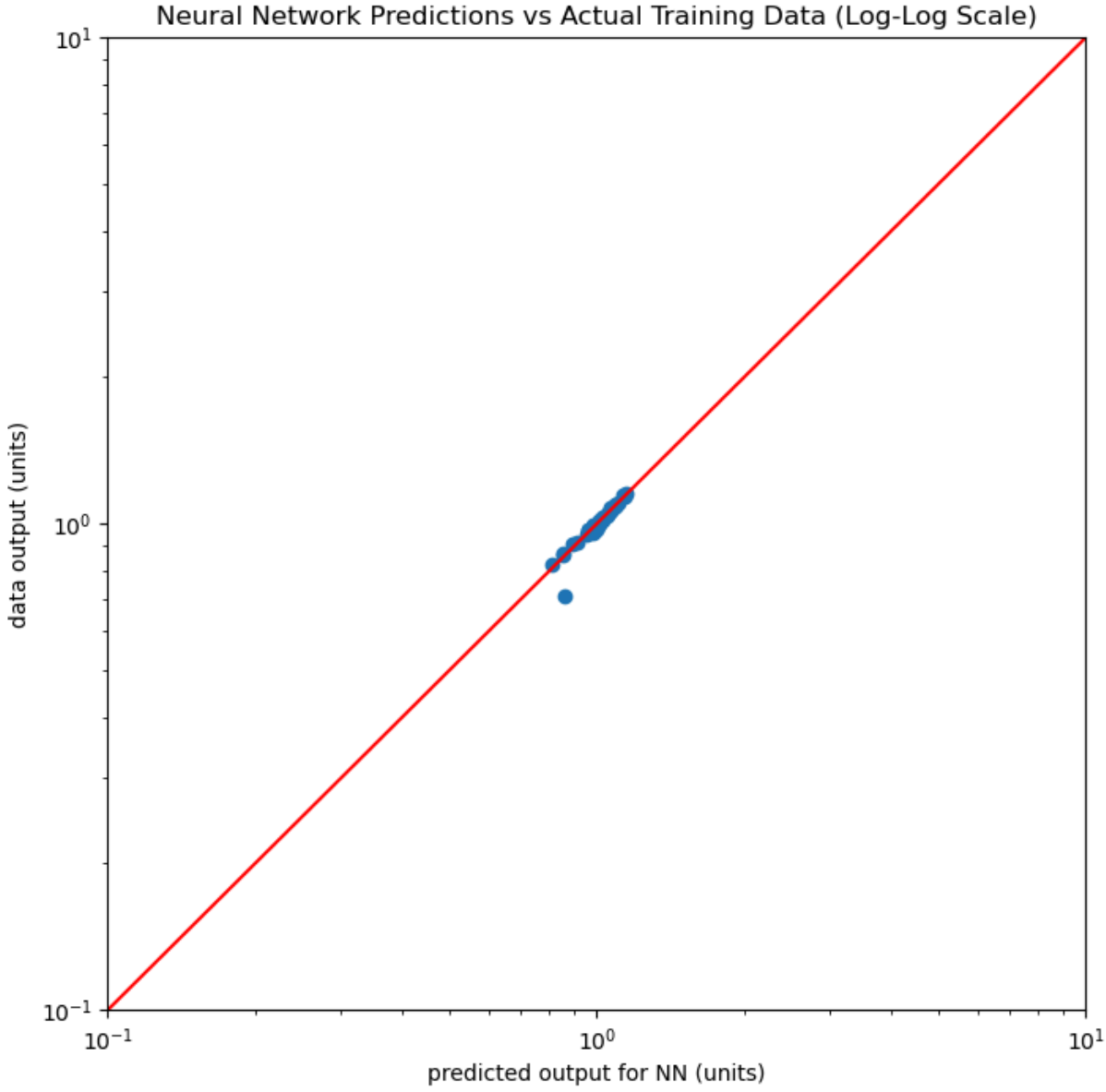
#put in a loop to print comparison for all data points
for i in range(len(x_train)):
    test = [[x_train[i][0], x_train[i][1], x_train[i][2]]]
    testarray = np.array(test)
    outpt = model.predict(testarray, verbose=0)
    print('row [' + str(i) + '] data:  T1= ', xarray[i][0], ', ID= ', xarray
        ', RL= ', xarray[i][2], ', power= ', yarray[i][1]*Wdmed, \
        ',  predicted power = ', outpt[0][1]*Wdmed)

#SETTING UP PLOT
%matplotlib inline
# importing the required module
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [8, 8] # for square canvas
#========
'''CALCULATE PREDICTED VALUES AND RETRIEVE DATA VALUES TO PLOT'''
# Denormalize for plotting
predictions_denorm = predictions[:, 0]
actual_denorm = y_train[:, 0]

plt.scatter(predictions_denorm, actual_denorm)
plt.title('Neural Network Predictions vs Actual Training Data (Log-Log Scale
plt.xlabel('predicted output for NN (units)')
plt.ylabel('data output (units)')
plt.loglog()
plt.xlim(xmax=10, xmin=0.1)
plt.ylim(ymax=10, ymin=0.1)
# Generate red y=x line using logspace for proper log scale
x_data = np.logspace(np.log10(0.1), np.log10(10.0), num=100)
y_data = x_data
plt.plot(x_data, y_data, color='red')
plt.show()
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step
row [0] data:  T1=  0.9293660603920184 , ID=  1.4090909090909092 , RL=  0.66
66666666666667 , power=  152.8 ,  predicted power =  119.56393432617188
row [1] data:  T1=  0.9293660603920184 , ID=  1.6818181818181819 , RL=  0.66
66666666666667 , power=  172.7 ,  predicted power =  96.31323432922363
row [2] data:  T1=  1.0 , ID=  1.4090909090909092 , RL=  0.6666666666666667
, power=  191.4 ,  predicted power =  89.25266790390015
row [3] data:  T1=  1.0 , ID=  1.6818181818181819 , RL=  0.6666666666666667
, power=  198.0 ,  predicted power =  91.35988306999207
row [4] data:  T1=  1.0706339396079816 , ID=  1.4090909090909092 , RL=  0.66
66666666666667 , power=  217.8 ,  predicted power =  80.77561569213867
row [5] data:  T1=  1.0706339396079816 , ID=  1.6818181818181819 , RL=  0.66
66666666666667 , power=  224.4 ,  predicted power =  221.85993146896362
row [6] data:  T1=  0.9293660603920184 , ID=  1.4090909090909092 , RL=  1.0
, power=  118.8 ,  predicted power =  121.90406608581543
row [7] data:  T1=  0.9293660603920184 , ID=  1.6818181818181819 , RL=  1.0
, power=  122.1 ,  predicted power =  99.52310943603516
row [8] data:  T1=  1.0 , ID=  1.4090909090909092 , RL=  0.9998506571087217
, power=  136.4 ,  predicted power =  199.5354356765747
row [9] data:  T1=  1.0 , ID=  1.6818181818181819 , RL=  1.0 , power=  139.6
,  predicted power =  99.896479845047
row [10] data:  T1=  1.0706339396079816 , ID=  1.4090909090909092 , RL=  1.0
, power=  154.0 ,  predicted power =  96.64077091217041
row [11] data:  T1=  1.0706339396079816 , ID=  1.6818181818181819 , RL=  1.0
002986857825569 , power=  158.4 ,  predicted power =  133.66494703292847
row [12] data:  T1=  0.9293660603920184 , ID=  1.4090909090909092 , RL=  1.3
333333333333335 , power=  92.4 ,  predicted power =  132.51202821731567
row [13] data:  T1=  0.9293660603920184 , ID=  1.6818181818181819 , RL=  1.3
333333333333335 , power=  94.6 ,  predicted power =  117.63363122940063
row [14] data:  T1=  1.0 , ID=  1.4090909090909092 , RL=  1.3333333333333335
, power=  105.6 ,  predicted power =  83.72400498390198
row [15] data:  T1=  1.0 , ID=  1.6818181818181819 , RL=  1.3333333333333335
, power=  107.8 ,  predicted power =  208.86313438415527
row [16] data:  T1=  1.0706339396079816 , ID=  1.4090909090909092 , RL=  1.3
333333333333335 , power=  119.9 ,  predicted power =  124.11004972457886
row [17] data:  T1=  1.0706339396079816 , ID=  1.6818181818181819 , RL=  1.3
333333333333335 , power=  123.20000000000002 ,  predicted power =  156.87347
078323364
row [18] data:  T1=  0.9293660603920184 , ID=  0.3181818181818182 , RL=  0.6
666666666666667 , power=  80.3 ,  predicted power =  86.28747034072876
row [19] data:  T1=  0.9293660603920184 , ID=  0.5909090909090909 , RL=  0.6
666666666666667 , power=  124.6 ,  predicted power =  109.19930839538574
row [20] data:  T1=  0.9293660603920184 , ID=  0.8636363636363636 , RL=  0.6
666666666666667 , power=  138.6 ,  predicted power =  77.33458018302917
row [21] data:  T1=  0.9293660603920184 , ID=  1.1363636363636365 , RL=  0.6
666666666666667 , power=  146.9 ,  predicted power =  108.6080527305603
row [22] data:  T1=  1.0 , ID=  0.3181818181818182 , RL=  0.6666666666666667
, power=  83.1 ,  predicted power =  143.39244890213013
row [23] data:  T1=  1.0 , ID=  0.5909090909090909 , RL=  0.6666666666666667
, power=  140.5 ,  predicted power =  115.02630949020386
row [24] data:  T1=  1.0 , ID=  0.8636363636363636 , RL=  0.6666666666666667
, power=  157.6 ,  predicted power =  75.99043703079224
row [25] data:  T1=  1.0 , ID=  1.1363636363636365 , RL=  0.6666666666666667
, power=  167.5 ,  predicted power =  113.3593521118164
row [26] data:  T1=  1.0706339396079816 , ID=  0.3181818181818182 , RL=  0.6
666666666666667 , power=  84.7 ,  predicted power =  172.69593811035156
```

```
row [27] data:  T1=  1.0706339396079816 , ID=  0.5909090909090909 , RL=  0.6
666666666666667 , power=  156.7 ,  predicted power =  162.0554656982422
row [28] data:  T1=  1.0706339396079816 , ID=  0.8636363636363636 , RL=  0.6
666666666666667 , power=  177.7 ,  predicted power =  164.3394684791565
row [29] data:  T1=  1.0706339396079816 , ID=  1.1363636363636365 , RL=  0.6
669653524492235 , power=  189.4 ,  predicted power =  80.73854899406433
row [30] data:  T1=  0.9293660603920184 , ID=  0.3181818181818182 , RL=  1.0
, power=  75.2 ,  predicted power =  156.15529918670654
row [31] data:  T1=  0.9293660603920184 , ID=  0.5909090909090909 , RL=  1.0
, power=  92.2 ,  predicted power =  88.15148758888245
row [32] data:  T1=  0.9293660603920184 , ID=  0.8636363636363636 , RL=  1.0
, power=  99.7 ,  predicted power =  128.048011302948
row [33] data:  T1=  0.9293660603920184 , ID=  1.1363636363636365 , RL=  1.0
, power=  104.6 ,  predicted power =  194.80186414718628
row [34] data:  T1=  1.0 , ID=  0.3181818181818182 , RL=  1.0 , power=  83.7
,  predicted power =  71.41076731681824
row [35] data:  T1=  1.0 , ID=  0.5909090909090909 , RL=  1.0 , power=  104.
9 ,  predicted power =  177.10385370254517
```



In [72]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# ===== FIXED PARAMETER =====
T_fixed_celsius = 20.0  # Fixed air temperature in Celsius
T_fixed_kelvin = T_fixed_celsius + 273.15  # Convert to Kelvin

print("===== Fixed Parameters =====")
print(f"Air Temperature: {T_fixed_celsius}°C = {T_fixed_kelvin:.2f} K")

# ===== CREATE GRID FOR PREDICTIONS =====
# Load resistance range: 4 to 8 Ohms
RL_min, RL_max = 4.0, 8.0
RL_points = 20  # Number of points in RL direction

# Irradiance range: 500 to 1200 W/m^2
ID_min, ID_max = 500.0, 1900.0
ID_points = 20  # Number of points in ID direction

# Create meshgrid
RL_array = np.linspace(RL_min, RL_max, RL_points)
ID_array = np.linspace(ID_min, ID_max, ID_points)
RL_mesh, ID_mesh = np.meshgrid(RL_array, ID_array)

print("===== Input Ranges =====")
print(f"Load Resistance (RL): {RL_min} - {RL_max} Ohms ({RL_points} points)"
print(f"Irradiance (ID): {ID_min} - {ID_max} W/m² ({ID_points} points)")
print(f"Total predictions: {RL_points * ID_points}\n")

# ===== NORMALIZE INPUT DATA FOR MODEL =====
# Flatten meshgrid for model prediction
RL_flat = RL_mesh.flatten()
ID_flat = ID_mesh.flatten()
# Fill with fixed Kelvin temperature
T_flat = np.full_like(RL_flat, T_fixed_kelvin)

# Normalize by medians (same way as training)
RL_norm = RL_flat / RLmed
ID_norm = ID_flat / IDmed
T_norm = T_flat / Tamed

# Create input array [T, ID, RL] for the model
input_data = np.column_stack([T_norm, ID_norm, RL_norm])

print("===== Normalized Input Statistics =====")
print(f"Temperature (normalized): {T_norm[0]:.4f}")
print(f"Irradiance (normalized): min={ID_norm.min():.4f}, max={ID_norm.max()
print(f"Load Resistance (normalized): min={RL_norm.min():.4f}, max={RL_norm.

# ===== GET PREDICTIONS FROM MODEL =====
predictions_normalized = model.predict(input_data, verbose=0)

# Extract power output (second column) and denormalize
power_normalized = predictions_normalized[:, 1]  # Column 1 is power output
```

```
power_denorm = power_normalized * Wdmed  # Denormalize using median

# Reshape back to grid
power_grid = power_denorm.reshape(RL_mesh.shape)

print("===== Power Output Statistics =====")
print(f"Minimum Power: {power_grid.min():.2f} W")
print(f"Maximum Power: {power_grid.max():.2f} W")
print(f"Mean Power: {power_grid.mean():.2f} W\n")

# ===== CREATE 3D SURFACE PLOT =====
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

# Create surface plot
surf = ax.plot_surface(RL_mesh, ID_mesh, power_grid, cmap='viridis',
                       edgecolor='none', alpha=0.9, antialiased=True)

# Labels and title
ax.set_xlabel('Load Resistance RL (Ohms)', fontsize=11, fontweight='bold')
ax.set_ylabel('Irradiance ID (W/m²)', fontsize=11, fontweight='bold')
ax.set_zlabel('Power Output Wd (Watts)', fontsize=11, fontweight='bold')
ax.set_title(f'Solar Power Output vs RL and ID\n(Fixed Temperature: {T_fixed
             fontsize=13, fontweight='bold')

# Add colorbar
cbar = fig.colorbar(surf, ax=ax, pad=0.1, shrink=0.8)
cbar.set_label('Power (W)', fontsize=11, fontweight='bold')

# Adjust viewing angle for better visualization
ax.view_init(elev=25, azim=45)

plt.savefig("surface_plot.png", dpi=300, bbox_inches='tight')
plt.show()

# ===== CREATE CONTOUR PLOT =====
fig, ax = plt.subplots(figsize=(10, 7))

# Create filled contour plot
contourf = ax.contourf(RL_mesh, ID_mesh, power_grid, levels=20, cmap='viridi
contour = ax.contour(RL_mesh, ID_mesh, power_grid, levels=10, colors='black'
                     alpha=0.3, linewidths=0.5)

# Add contour labels
ax.clabel(contour, inline=True, fontsize=9, fmt='%.1f W')

# Labels and title
ax.set_xlabel('Load Resistance RL (Ohms)', fontsize=11, fontweight='bold')
ax.set_ylabel('Irradiance ID (W/m²)', fontsize=11, fontweight='bold')
ax.set_title(f'Power Output Contour Map\n(Fixed Temperature: {T_fixed_celsiu
             fontsize=13, fontweight='bold')

# Add colorbar
cbar = fig.colorbar(contourf, ax=ax)
cbar.set_label('Power (W)', fontsize=11, fontweight='bold')
plt.savefig("Contour_plot.png", dpi=300, bbox_inches='tight')
```

```
plt.show()

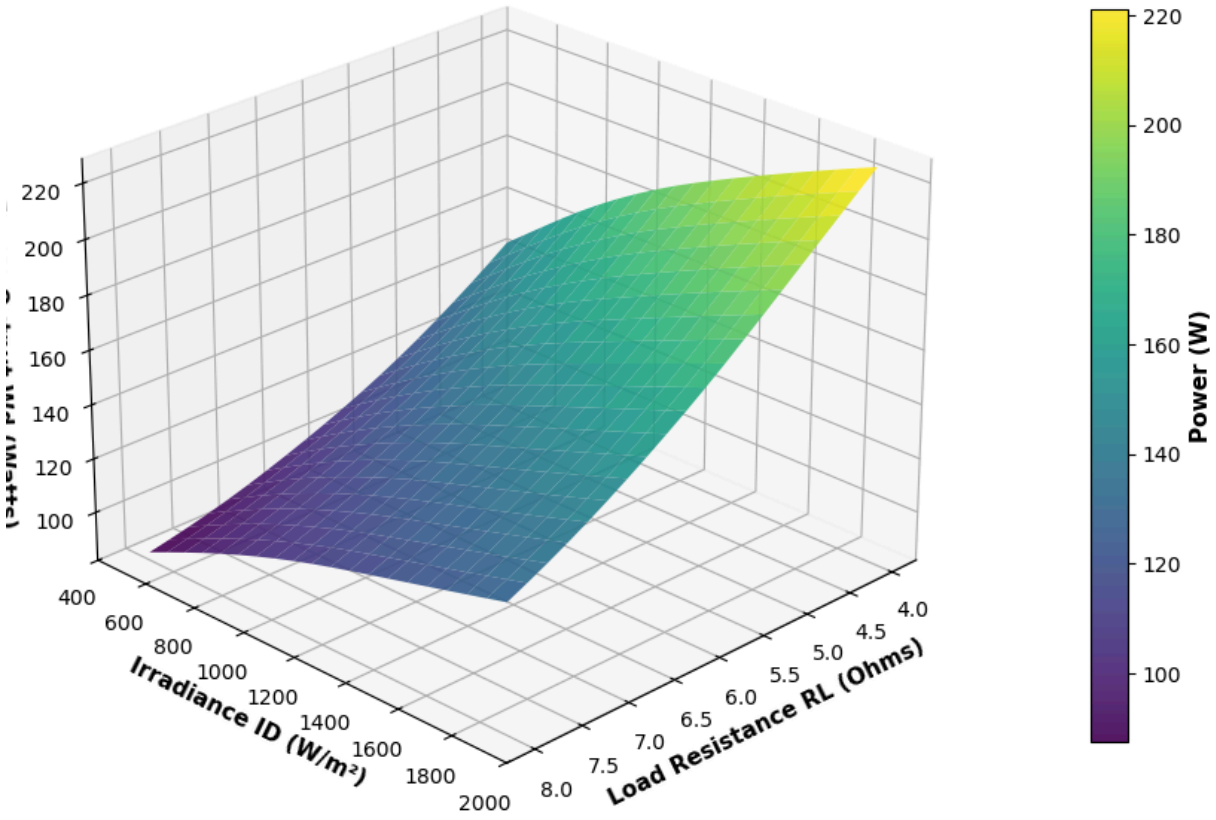
# ===== PRINT SAMPLE PREDICTIONS =====
print("===== Sample Predictions =====\n")
sample_indices = [0, RL_points-1, RL_points*ID_points//2, RL_points*ID_point
for idx in sample_indices:
    if idx < len(RL_flat):
        print(f"RL = {RL_flat[idx]:.2f} Ω, ID = {ID_flat[idx]:.0f} W/m², Pow

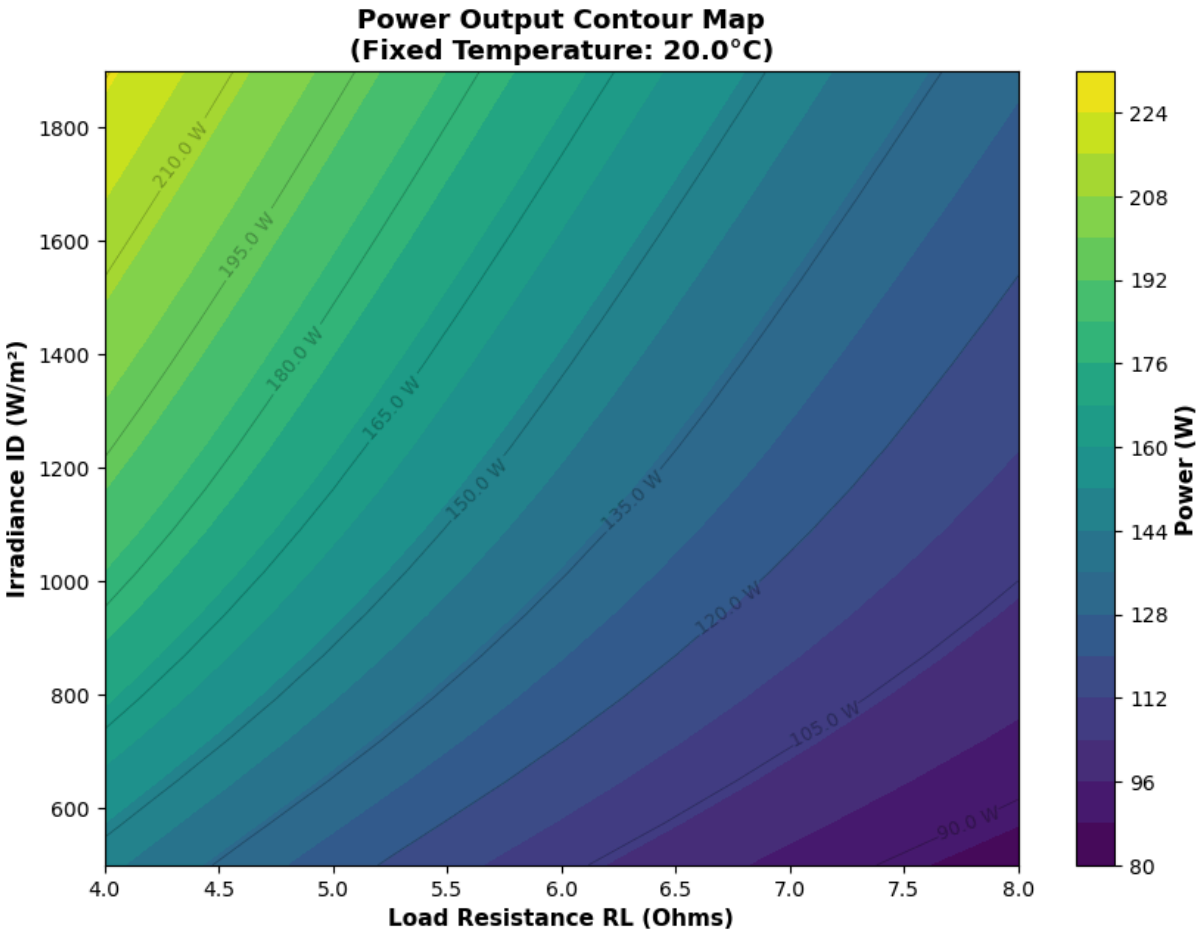
# ===== OPTIONAL: Find optimal RL for maximum power at each ID =====
print("\n===== Optimal Load Resistance for Maximum Power =====\n")
print("ID (W/m²)  |  Optimal RL (Ω)  |  Max Power (W)")
print("-" * 50)
for j in range(0, ID_points, ID_points//5):
    max_power_idx = np.argmax(power_grid[j, :])
    optimal_rl = RL_array[max_power_idx]
    max_power = power_grid[j, max_power_idx]
    print(f"{ID_array[j]:6.0f}     |     {optimal_rl:6.2f}      |     {max_p
```

```
===== Fixed Parameters =====
Air Temperature: 20.0°C = 293.15 K
===== Input Ranges =====
Load Resistance (RL): 4.0 - 8.0 Ohms (20 points)
Irradiance (ID): 500.0 - 1900.0 W/m² (20 points)
Total predictions: 400

===== Normalized Input Statistics =====
Temperature (normalized): 1.0353
Irradiance (normalized): min=0.4545, max=1.7273
Load Resistance (normalized): min=0.5974, max=1.1947

===== Power Output Statistics =====
Minimum Power: 84.96 W
Maximum Power: 225.73 W
Mean Power: 145.03 W
```


Solar Power Output vs RL and ID
(Fixed Temperature: 20.0°C)
Irradiance ID (W/m²)
Load Resistance RL (Ohms)
Power (W)
100
120
140
160
180
200
220
400
600
800
1000
1200
1400
1600
1800
2000
4.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0




```
===== Sample Predictions =====

RL = 4.00 Ω, ID = 500 W/m², Power = 146.14 W
RL = 8.00 Ω, ID = 500 W/m², Power = 84.96 W
RL = 4.00 Ω, ID = 1237 W/m², Power = 195.94 W
RL = 8.00 Ω, ID = 1900 W/m², Power = 129.43 W

===== Optimal Load Resistance for Maximum Power =====

ID (W/m²)  |  Optimal RL (Ω)  |  Max Power (W)
-------------------------------------------------
   500     |       4.00       |      146.14
   795     |       4.00       |      169.14
  1089     |       4.00       |      188.13
  1384     |       4.00       |      203.02
  1679     |       4.00       |      216.29
```

In [ ]:

In [ ]:

```
'''>>>>> start CodeP3.2F25
    V.P. Carey ME249, Fall 2025

Intro to Neural Network Modeling
Keras model for PV power system'''

#import useful packages
import keras
import pandas as pd
from keras.models import Sequential
import numpy as np
import keras.backend as kb
import tensorflow as tf
#the following 2 lines are only needed for Mac OS machines
import os
os.environ['KMP_DUPLICATE_LIB_OK']='True'


# ===== PART 1: Define raw data first, convert temp to Kelvin, then calculat
# Raw input data: Mode number, Air temp (degC), ID (W/sqm), load resistance
xdata_raw = [[0., 10.0, 200.0, 24.3],
 [1., 10.0, 200.0, 24.3],
 [2., 10.0, 200.0, 24.3],
 [0., 10.0, 200.0, 51.8],
 [1., 10.0, 200.0, 51.8],
 [2., 10.0, 200.0, 51.8],
 [0., 10.0, 200.0, 96.2],
 [1., 10.0, 200.0, 96.2],
 [2., 10.0, 200.0, 96.1],
 [0., 10.0, 200.0, 170.1],
 [1., 10.0, 200.0, 170.1],
 [2., 10.0, 200.0, 170.1],
 [0., 10.0, 500.0, 7.0],
 [1., 10.0, 500.0, 7.0],
 [2., 10.0, 500.0, 7.0],
 [0., 10.0, 500.0, 21.2],
 [1., 10.0, 500.0, 21.2],
 [2., 10.0, 500.0, 21.2],
 [0., 10.0, 500.0, 43.2],
 [1., 10.0, 500.0, 43.2],
 [1., 10.0, 700.0, 4.9],
 [2., 10.0, 700.0, 4.9],
 [0., 10.0, 700.0, 14.3],
 [1., 10.0, 700.0, 14.3],
 [2., 10.0, 700.0, 14.3],
 [2., 10.0, 500.0, 43.2],
 [0., 10.0, 500.0, 79.1],
 [1., 10.0, 500.0, 79.1],
 [2., 10.0, 500.0, 79.1],
 [0., 10.0, 700.0, 4.8],
 [0., 10.0, 700.0, 29.7],
 [1., 10.0, 700.0, 29.7],
 [2., 10.0, 700.0, 29.7],
 [0., 10.0, 700.0, 55.3],
```

```
 [1., 10.0, 700.0, 55.3],
 [2., 10.0, 700.0, 55.3],
 [0., 10.0, 1000.0, 3.92],
 [1., 10.0, 1000.0, 3.92],
 [2., 10.0, 1000.0, 3.92],
 [0., 10.0, 1000.0, 11.7],
 [1., 10.0, 1000.0, 11.7],
 [2., 10.0, 1000.0, 11.7],
 [0., 10.0, 1000.0, 25.2],
 [1., 10.0, 1000.0, 25.2],
 [2., 10.0, 1000.0, 25.2],
 [0., 10.0, 1000.0, 41.6],
 [1., 10.0, 1000.0, 41.6],
 [2., 10.0, 1000.0, 41.6]]

#Part 1 output data: VL (V) and Power out (W)
ydata_raw = [[46.0, 87.3],
 [27.9, 64.2],
 [27.9, 32.1],
 [46.0, 40.9],
 [92.1, 163.9],
 [59.5, 68.5],
 [46.0, 22.0],
 [92.1, 88.2],
 [110.6, 127.2],
 [46.0, 12.4],
 [92.1, 49.9],
 [184.2, 199.6],
 [48.4, 335.3],
 [20.8, 124.3],
 [20.8, 62.1],
 [48.4, 110.7],
 [96.8, 442.9],
 [63.1, 188.2],
 [48.4, 54.3],
 [96.9, 217.3],
 [128.7, 383.6],
 [48.4, 29.6],
 [96.8, 118.7],
 [193.7, 474.8],
 [49.3, 496.1],
 [20.4, 171.2],
 [20.4, 85.6],
 [49.3, 169.9],
 [98.6, 679.9],
 [59.7, 249.8],
 [49.3, 81.8],
 [98.6, 327.4],
 [124.1, 518.9],
 [49.305, 43.9],
 [98.6, 175.8],
 [197.2, 703.3],
 [50.8, 658.9],
 [23.1, 272.9],
 [23.1, 136.4],
 [50.8, 220.7],
```

```
 [101.6, 883.1],
 [69.0, 407.2],
 [50.8, 102.5],
 [101.6, 410.0],
 [148.6, 877.2],
 [50.8, 62.0],
 [101.6, 248.3],
 [203.2, 993.5]]

# Convert temperature from Celsius to Kelvin (K = degC + 273.15)
# Keep Mode (column 0) unchanged, convert Temp (column 1) to Kelvin
xdata_kelvin = [[row[0], row[1] + 273.15, row[2], row[3]] for row in xdata_r

print("===== Temperature Conversion =====")
print("First 3 samples (Celsius -> Kelvin):")
for i in range(3):
    print(f"  Mode={xdata_kelvin[i][0]:.1f}, Temp: {xdata_raw[i][1]:6.2f} °C
print()

# Convert to numpy arrays and calculate medians
xarray = np.array(xdata_kelvin)
yarray = np.array(ydata_raw)

# Calculate input variable medians from data
Tamed = np.median(xarray[:, 1])      # Air temperature median (K)
IDmed = np.median(xarray[:, 2])      # Irradiance (ID) median (W/sqm)
RLmed = np.median(xarray[:, 3])      # Load resistance median (ohms)

# Calculate output variable medians from data
VLmed = np.median(yarray[:, 0])      # Load voltage (VL) median (V)
Wdmed = np.median(yarray[:, 1])      # Power output (Wd) median (W)

print("===== Calculated Median Values =====")
print(f"Temperature median: {Tamed:.2f} K")
print(f"Irradiance median: {IDmed:.2f} W/sqm")
print(f"Load resistance median: {RLmed:.4f} ohms")
print(f"Voltage median: {VLmed:.2f} V")
print(f"Power median: {Wdmed:.2f} W\n")

# ===== PART 2: Create normalized input data array =====
# Normalize input data by dividing by medians
# Mode stays unnormalized (keep as is), Temperature/ID/RL normalized
xdata = []
for row in xdata_kelvin:
    xdata.append([row[0], row[1]/Tamed, row[2]/IDmed, row[3]/RLmed])

xarray = np.array(xdata)
print("===== Normalized Input Data =====")
print("First 5 rows (normalized):")
print(xarray[:5])
print(f"Input shape: {xarray.shape}\n")

# ===== PART 3: Create normalized output data array =====
# Normalize output data by dividing by medians
ydata = []
for row in ydata_raw:
```

```
    ydata.append([row[0]/VLmed, row[1]/Wdmed])

yarray = np.array(ydata)
print("===== Normalized Output Data =====")
print("First 5 rows (normalized):")
print(yarray[:5])
print(f"Output shape: {yarray.shape}\n")

# ===== PART 4: Split data into training (2/3) and validation (1/3) sets ===
# Set random seed for reproducibility
np.random.seed(42)

# Generate random indices and shuffle them
total_samples = len(xarray)
indices = np.arange(total_samples)
np.random.shuffle(indices)

# Calculate split point (2/3 for training, 1/3 for validation)
split_point = int(2/3 * total_samples)

# Split indices
train_indices = indices[:split_point]
val_indices = indices[split_point:]

# Create training and validation sets
x_train = xarray[train_indices]
y_train = yarray[train_indices]
x_val = xarray[val_indices]
y_val = yarray[val_indices]

print("===== Data Split =====")
print(f"Total samples: {total_samples}")
print(f"Training set size: {len(x_train)} ({len(x_train)/total_samples*100:.
print(f"Validation set size: {len(x_val)} ({len(x_val)/total_samples*100:.1f
print(f"\nTraining input shape: {x_train.shape}")
print(f"Training output shape: {y_train.shape}")
print(f"Validation input shape: {x_val.shape}")
print(f"Validation output shape: {y_val.shape}\n")

print("===== Median Values Summary =====")
print(f"Tamed = {Tamed:.2f}")
print(f"IDmed = {IDmed:.2f}")
print(f"RLmed = {RLmed:.4f}")
print(f"VLmed = {VLmed:.2f}")
print(f"Wdmed = {Wdmed:.2f}")
```

In [52]:

```
# define neural network model

#As seen below, we have created four dense layers.
#A dense layer is a layer in neural network that's fully connected.
#In other words, all the neurons in one layer are connected to all other neu
#In the first layer, we need to provide the input shape, which is 1 in our c
#The activation function we have chosen is elu, which stands for exponential

from keras import backend as K
#initialize weights with values between -0.2 and 0.5
```

```
initializer = keras.initializers.RandomUniform(minval= -0.2, maxval=0.5)

model = keras.Sequential([
    keras.layers.Dense(13, activation='elu', input_shape=[4],  kernel_initia
    keras.layers.Dense(26, activation='elu',  kernel_initializer=initializer
    keras.layers.Dense(13, activation='elu', kernel_initializer=initializer)
    keras.layers.Dense(2,  kernel_initializer=initializer)
  ])
```

In [98]:
```
#We're using RMSprop as our optimizer here. RMSprop stands for Root Mean Squ
#It's one of the most popular gradient descent optimization algorithms for d
#RMSprop is an optimizer that's reliable and fast.
#We're compiling the mode using the model.compile function. The loss functio
#is mean squared error. After the compilation of the model, we'll use the fi
#Number of epochs can be varied.

#from tf.keras import optimizers
rms = keras.optimizers.RMSprop(0.00005)
model.compile(loss='mean_absolute_error',optimizer=rms)
```

In [103…
```
#After the compilation of the model, we'll use the fit method with 500 epoch
#I started with epochs value of 100 and then tested the model after training
#The prediction was not that good. Then I modified the number of epochs to 2
#Accuracy had improved slightly, but figured I'd give it one more try. Final
#I found acceptable prediction accuracy.

#The fit method takes three parameters; namely, x, y, and number of epochs.
#During model training, if all the batches of data are seen by the model onc
#we say that one epoch has been completed.

# Add an early stopping callback
es = keras.callbacks.EarlyStopping(
    monitor='loss',
    mode='min',
    patience = 80,
    restore_best_weights = True,
    verbose=1)
# Add a checkpoint where loss is minimum, and save that model
mc = keras.callbacks.ModelCheckpoint('best_model.keras', monitor='loss',
                     mode='min',  verbose=1, save_best_only=True)

historyData = model.fit(x_train,y_train,epochs=800,callbacks=[es])

loss_hist = historyData.history['loss']
#The above line will return a dictionary, access it's info like this:
best_epoch = np.argmin(historyData.history['loss']) + 1
print ('best epoch = ', best_epoch)
print('smallest loss =', np.min(loss_hist))

model.save('./best_model.keras')
```

```
Epoch 1/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - loss: 0.0340
Epoch 2/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - loss: 0.0334
Epoch 3/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0335
Epoch 4/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0336
Epoch 5/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0338
Epoch 6/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0333
Epoch 7/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0335
Epoch 8/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0340
Epoch 9/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0338
Epoch 10/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0339
Epoch 11/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0336
Epoch 12/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 13/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0335
Epoch 14/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0341
Epoch 15/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 16/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0342
Epoch 17/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0336
Epoch 18/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0340
Epoch 19/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0342
Epoch 20/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0342
Epoch 21/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 22/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0332
Epoch 23/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0334
Epoch 24/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0333
Epoch 25/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0340
Epoch 26/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0334
Epoch 27/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0340
Epoch 28/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0334
```

```
Epoch 29/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0341
Epoch 30/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 31/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0345
Epoch 32/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 33/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0337
Epoch 34/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0339
Epoch 35/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0340
Epoch 36/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0336
Epoch 37/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0339
Epoch 38/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0335
Epoch 39/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0337
Epoch 40/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0338
Epoch 41/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0334
Epoch 42/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0335
Epoch 43/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0335
Epoch 44/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0339
Epoch 45/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0334
Epoch 46/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0333
Epoch 47/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0334
Epoch 48/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0336
Epoch 49/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 50/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0339
Epoch 51/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0343
Epoch 52/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0338
Epoch 53/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0335
Epoch 54/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0340
Epoch 55/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 56/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0337
```

```
Epoch 57/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0338
Epoch 58/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0338
Epoch 59/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0335
Epoch 60/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0335
Epoch 61/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0340
Epoch 62/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0333
Epoch 63/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0334
Epoch 64/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0343
Epoch 65/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0339
Epoch 66/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0337
Epoch 67/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 68/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0339
Epoch 69/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0338
Epoch 70/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0336
Epoch 71/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0336
Epoch 72/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0335
Epoch 73/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0340
Epoch 74/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0335
Epoch 75/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 76/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0334
Epoch 77/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0335
Epoch 78/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0335
Epoch 79/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0339
Epoch 80/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0336
Epoch 81/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0340
Epoch 82/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0336
Epoch 83/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0336
Epoch 84/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0339
```

```
Epoch 85/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0341
Epoch 86/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0336
Epoch 87/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0334
Epoch 88/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0335
Epoch 89/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0335
Epoch 90/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0336
Epoch 91/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0338
Epoch 92/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 93/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0339
Epoch 94/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 95/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0336
Epoch 96/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0332
Epoch 97/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 98/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0335
Epoch 99/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0338
Epoch 100/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0336
Epoch 101/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - loss: 0.0337
Epoch 102/800
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0342
Epoch 102: early stopping
Restoring model weights from the end of the best epoch: 22.
best epoch =  22
smallest loss = 0.033173635601997375
```

In [104…

```
# This line of code can be used to reconstruct the saved model.

recon_model = keras.models.load_model("best_model.keras")

# the name of the model is now "recon_model". You can then use this model to
# See the previous project for code to do the comparisons.
```

In [106…

```
test = []
outpt=[]
predictions = model.predict(x_train, verbose=0)

# put in a loop to print comparison for all data points
for i in range(len(x_train)):
    # Use ALL 4 columns from training set
    test = [[x_train[i][0], x_train[i][1], x_train[i][2], x_train[i][3]]]
```

```
    testarray = np.array(test)
    outpt = model.predict(testarray, verbose=0)

    # Denormalize for display
    mode = x_train[i][0]
    temp = x_train[i][1] * Tamed
    irrad = x_train[i][2] * IDmed
    rl = x_train[i][3] * RLmed
    actual_power = y_train[i][1] * Wdmed
    pred_power = outpt[0][1] * Wdmed

    print(f'row [{i}] M={mode:.0f}  T={temp:.1f}K  ID={irrad:.1f}  RL={rl:.2
          f'actual_power={actual_power:.1f}W  predicted_power={pred_power:.1

# SETTING UP PLOT
%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [8, 8]

# ========
# CALCULATE PREDICTED VALUES AND RETRIEVE DATA VALUES TO PLOT
# Denormalize for plotting (use power: column 1)
predictions_denorm = predictions[:, 1] * Wdmed
actual_denorm = y_train[:, 1] * Wdmed

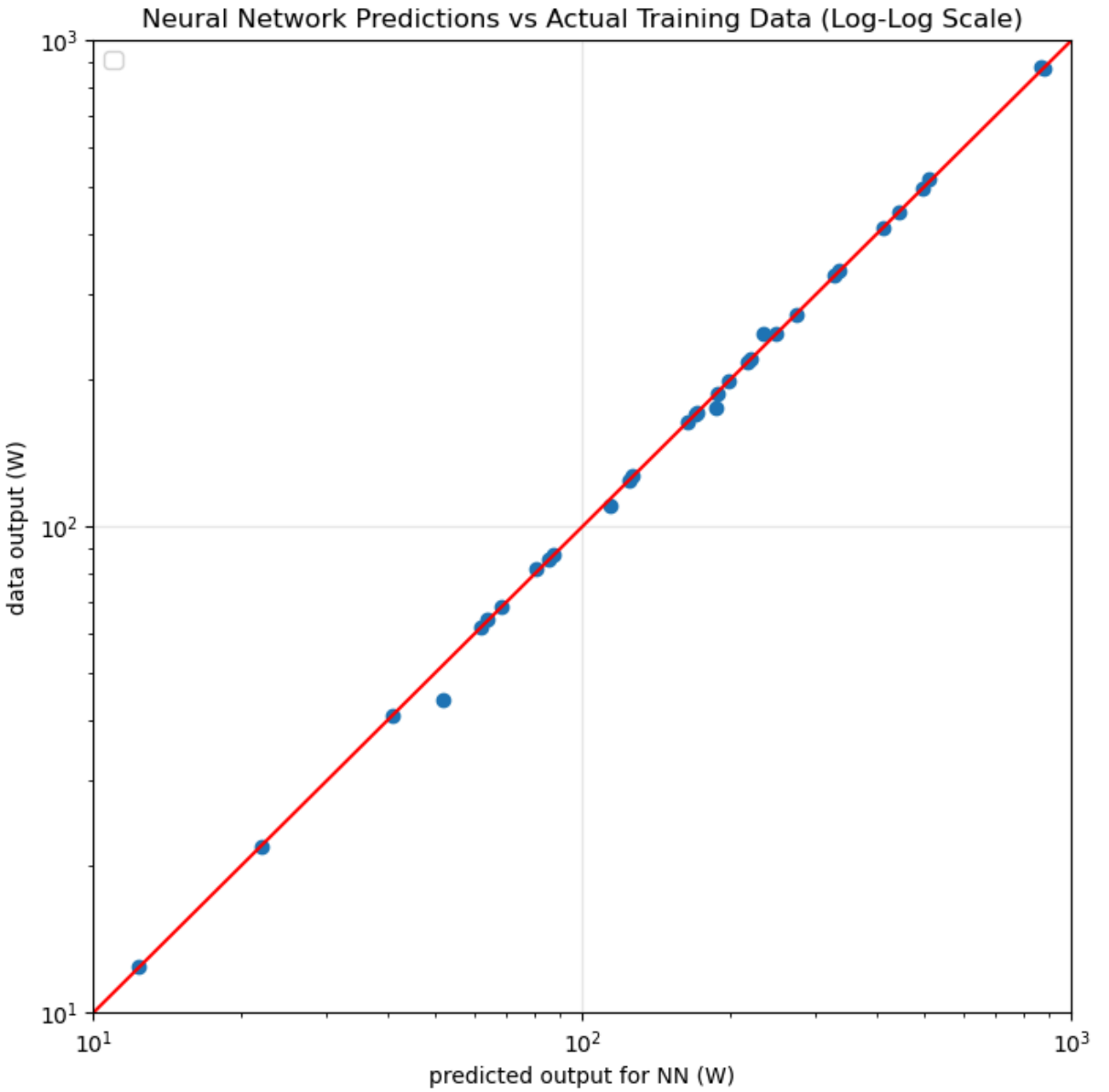
plt.scatter(predictions_denorm, actual_denorm)
plt.title('Neural Network Predictions vs Actual Training Data (Log-Log Scale
plt.xlabel('predicted output for NN (W)')
plt.ylabel('data output (W)')
plt.loglog()
plt.xlim(xmax=1000, xmin=10)
plt.ylim(ymax=1000, ymin=10)

# Generate red y=x line using logspace for proper log scale
x_data = np.logspace(np.log10(10.0), np.log10(1000.0), num=100)
y_data = x_data
plt.plot(x_data, y_data, color='red')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
row [0] M=1  T=283.1K  ID=500.0  RL=79.10  actual_power=169.9W  predicted_po
wer=170.0W
row [1] M=1  T=283.1K  ID=1000.0  RL=11.70  actual_power=883.1W  predicted_p
ower=868.5W
row [2] M=0  T=283.1K  ID=500.0  RL=79.10  actual_power=85.6W  predicted_pow
er=85.5W
row [3] M=1  T=283.1K  ID=1000.0  RL=25.20  actual_power=410.0W  predicted_p
ower=410.5W
row [4] M=2  T=283.1K  ID=700.0  RL=14.30  actual_power=496.1W  predicted_po
wer=496.3W
row [5] M=1  T=283.1K  ID=1000.0  RL=3.92  actual_power=272.9W  predicted_po
wer=273.4W
row [6] M=0  T=283.1K  ID=500.0  RL=7.00  actual_power=335.3W  predicted_pow
er=335.5W
row [7] M=1  T=283.1K  ID=500.0  RL=43.20  actual_power=217.3W  predicted_po
wer=216.9W
row [8] M=1  T=283.1K  ID=200.0  RL=51.80  actual_power=163.9W  predicted_po
wer=164.5W
row [9] M=2  T=283.1K  ID=500.0  RL=43.20  actual_power=171.2W  predicted_po
wer=171.4W
row [10] M=2  T=283.1K  ID=200.0  RL=96.10  actual_power=127.2W  predicted_p
ower=126.7W
row [11] M=0  T=283.1K  ID=200.0  RL=51.80  actual_power=40.9W  predicted_po
wer=40.9W
row [12] M=0  T=283.1K  ID=200.0  RL=96.20  actual_power=22.0W  predicted_po
wer=22.0W
row [13] M=0  T=283.1K  ID=1000.0  RL=11.70  actual_power=220.7W  predicted_
power=220.9W
row [14] M=0  T=283.1K  ID=700.0  RL=55.30  actual_power=43.9W  predicted_po
wer=51.8W
row [15] M=1  T=283.1K  ID=500.0  RL=7.00  actual_power=124.3W  predicted_po
wer=124.1W
row [16] M=2  T=283.1K  ID=500.0  RL=21.20  actual_power=188.2W  predicted_p
ower=188.9W
row [17] M=0  T=283.1K  ID=1000.0  RL=41.60  actual_power=62.0W  predicted_p
ower=62.1W
row [18] M=0  T=283.1K  ID=500.0  RL=21.20  actual_power=110.7W  predicted_p
ower=114.1W
row [19] M=0  T=283.1K  ID=200.0  RL=170.10  actual_power=12.4W  predicted_p
ower=12.3W
row [20] M=1  T=283.1K  ID=500.0  RL=21.20  actual_power=442.9W  predicted_p
ower=443.3W
row [21] M=0  T=283.1K  ID=700.0  RL=4.80  actual_power=249.8W  predicted_po
wer=249.2W
row [22] M=2  T=283.1K  ID=700.0  RL=29.70  actual_power=518.9W  predicted_p
ower=510.8W
row [23] M=1  T=283.1K  ID=1000.0  RL=41.60  actual_power=248.3W  predicted_
power=234.6W
row [24] M=0  T=283.1K  ID=200.0  RL=24.30  actual_power=87.3W  predicted_po
wer=87.3W
row [25] M=1  T=283.1K  ID=700.0  RL=29.70  actual_power=327.4W  predicted_p
ower=327.6W
row [26] M=0  T=283.1K  ID=700.0  RL=29.70  actual_power=81.8W  predicted_po
wer=80.3W
row [27] M=2  T=283.1K  ID=200.0  RL=51.80  actual_power=68.5W  predicted_po
wer=68.3W
```

```
row [28] M=2  T=283.1K  ID=200.0  RL=170.10  actual_power=199.6W  predicted_
power=199.1W
row [29] M=1  T=283.1K  ID=700.0  RL=55.30  actual_power=175.8W  predicted_p
ower=187.0W
row [30] M=1  T=283.1K  ID=200.0  RL=24.30  actual_power=64.2W  predicted_po
wer=63.9W
row [31] M=2  T=283.1K  ID=1000.0  RL=25.20  actual_power=877.2W  predicted_
power=878.4W
```

```
/tmp/ipykernel_1019621/154440754.py:46: UserWarning: No artists with labels
found to put in legend.  Note that artists whose label start with an undersc
ore are ignored when legend() is called with no argument.
  plt.legend()
```



In [108…

```
test = []
outpt=[]

# predictions on validation set
predictions = model.predict(x_val, verbose=0)

# put in a loop to print comparison for all data points
```

```
for i in range(len(xarray)):
    # Use ALL 4 columns: Mode, T_normalized, ID_normalized, RL_normalized
    test = [[xarray[i][0], xarray[i][1], xarray[i][2], xarray[i][3]]]
    testarray = np.array(test)
    outpt = model.predict(testarray, verbose=0)

    # Denormalize for display
    mode = xarray[i][0]
    temp = xarray[i][1] * Tamed
    irrad = xarray[i][2] * IDmed
    rl = xarray[i][3] * RLmed
    actual_power = yarray[i][1] * Wdmed
    pred_power = outpt[0][1] * Wdmed  # output index 1 is power

    print(f'row [{i}] M={mode:.0f}  T={temp:.1f}K  ID={irrad:.1f}  RL={rl:.2
          f'actual_power={actual_power:.1f}W  predicted_power={pred_power:.1

# SETTING UP PLOT
%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [8, 8]

# ========
# CALCULATE PREDICTED VALUES AND RETRIEVE DATA VALUES TO PLOT
# Denormalize for plotting (output index 1 is power)
predictions_denorm = predictions[:, 1] * Wdmed  # Power is column 1
actual_denorm = y_val[:, 1] * Wdmed             # Use validation data power

plt.scatter(predictions_denorm, actual_denorm)
plt.title('Neural Network Predictions vs Actual Validation Data (Log-Log Sca
plt.xlabel('predicted output for NN (W)')
plt.ylabel('data output (W)')
plt.loglog()
plt.xlim(xmax=1000, xmin=10)
plt.ylim(ymax=1000, ymin=10)

# Generate red y=x line using logspace for proper log scale
x_data = np.logspace(np.log10(10.0), np.log10(1000.0), num=100)
y_data = x_data
plt.plot(x_data, y_data, color='red')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()


# Calculate Mean Absolute Error (MAE) for validation data
mae = np.mean(np.abs(predictions_denorm - actual_denorm))

print(f"\n===== Validation Data Error Analysis =====")
print(f"Mean Absolute Error (MAE): {mae:.4f} W")
print(f"MAE as % of mean power: {(mae / np.mean(actual_denorm)) * 100:.2f}%"

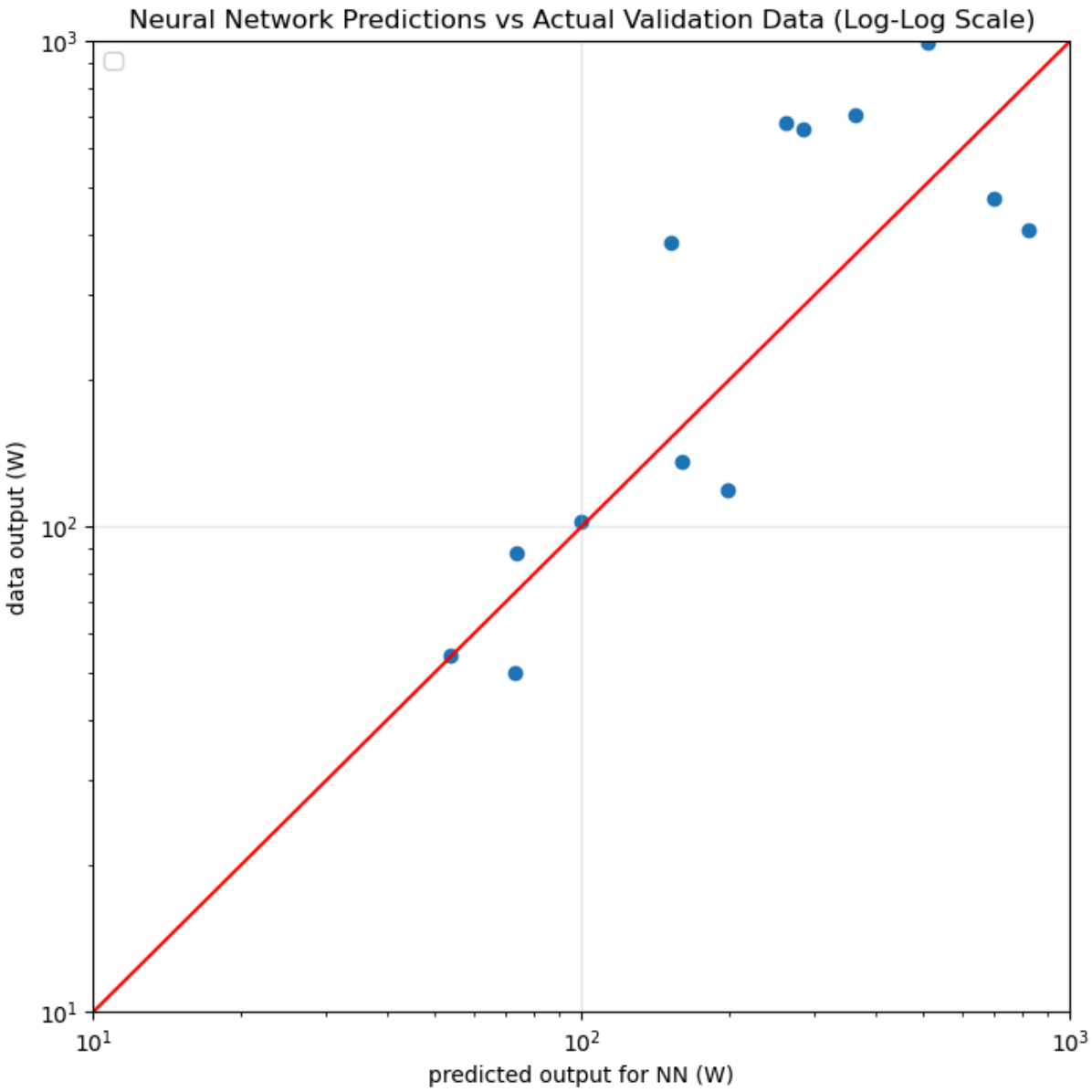
# Additional useful metrics
rmse = np.sqrt(np.mean((predictions_denorm - actual_denorm)**2))
print(f"Root Mean Squared Error (RMSE): {rmse:.4f} W")
```

```
max_error = np.max(np.abs(predictions_denorm - actual_denorm))
min_error = np.min(np.abs(predictions_denorm - actual_denorm))
print(f"Max Absolute Error: {max_error:.4f} W")
print(f"Min Absolute Error: {min_error:.4f} W")

# R-squared (coefficient of determination)
ss_res = np.sum((actual_denorm - predictions_denorm)**2)
ss_tot = np.sum((actual_denorm - np.mean(actual_denorm))**2)
r_squared = 1 - (ss_res / ss_tot)
print(f"R-squared (R²): {r_squared:.4f}")
```

```
row [0] M=0  T=283.1K  ID=200.0  RL=24.30  actual_power=87.3W  predicted_pow
er=87.3W
row [1] M=1  T=283.1K  ID=200.0  RL=24.30  actual_power=64.2W  predicted_pow
er=63.9W
row [2] M=2  T=283.1K  ID=200.0  RL=24.30  actual_power=32.1W  predicted_pow
er=-208.8W
row [3] M=0  T=283.1K  ID=200.0  RL=51.80  actual_power=40.9W  predicted_pow
er=40.9W
row [4] M=1  T=283.1K  ID=200.0  RL=51.80  actual_power=163.9W  predicted_po
wer=164.5W
row [5] M=2  T=283.1K  ID=200.0  RL=51.80  actual_power=68.5W  predicted_pow
er=68.3W
row [6] M=0  T=283.1K  ID=200.0  RL=96.20  actual_power=22.0W  predicted_pow
er=22.0W
row [7] M=1  T=283.1K  ID=200.0  RL=96.20  actual_power=88.2W  predicted_pow
er=73.6W
row [8] M=2  T=283.1K  ID=200.0  RL=96.10  actual_power=127.2W  predicted_po
wer=126.7W
row [9] M=0  T=283.1K  ID=200.0  RL=170.10  actual_power=12.4W  predicted_po
wer=12.3W
row [10] M=1  T=283.1K  ID=200.0  RL=170.10  actual_power=49.9W  predicted_p
ower=73.1W
row [11] M=2  T=283.1K  ID=200.0  RL=170.10  actual_power=199.6W  predicted_
power=199.1W
row [12] M=0  T=283.1K  ID=500.0  RL=7.00  actual_power=335.3W  predicted_po
wer=335.5W
row [13] M=1  T=283.1K  ID=500.0  RL=7.00  actual_power=124.3W  predicted_po
wer=124.1W
row [14] M=2  T=283.1K  ID=500.0  RL=7.00  actual_power=62.1W  predicted_pow
er=-198.0W
row [15] M=0  T=283.1K  ID=500.0  RL=21.20  actual_power=110.7W  predicted_p
ower=114.1W
row [16] M=1  T=283.1K  ID=500.0  RL=21.20  actual_power=442.9W  predicted_p
ower=443.3W
row [17] M=2  T=283.1K  ID=500.0  RL=21.20  actual_power=188.2W  predicted_p
ower=188.9W
row [18] M=0  T=283.1K  ID=500.0  RL=43.20  actual_power=54.3W  predicted_po
wer=53.8W
row [19] M=1  T=283.1K  ID=500.0  RL=43.20  actual_power=217.3W  predicted_p
ower=216.9W
row [20] M=1  T=283.1K  ID=700.0  RL=4.90  actual_power=383.6W  predicted_po
wer=152.7W
row [21] M=2  T=283.1K  ID=700.0  RL=4.90  actual_power=29.6W  predicted_pow
er=-76.8W
row [22] M=0  T=283.1K  ID=700.0  RL=14.30  actual_power=118.7W  predicted_p
ower=198.9W
row [23] M=1  T=283.1K  ID=700.0  RL=14.30  actual_power=474.8W  predicted_p
ower=695.6W
row [24] M=2  T=283.1K  ID=700.0  RL=14.30  actual_power=496.1W  predicted_p
ower=496.3W
row [25] M=2  T=283.1K  ID=500.0  RL=43.20  actual_power=171.2W  predicted_p
ower=171.4W
row [26] M=0  T=283.1K  ID=500.0  RL=79.10  actual_power=85.6W  predicted_po
wer=85.5W
row [27] M=1  T=283.1K  ID=500.0  RL=79.10  actual_power=169.9W  predicted_p
ower=170.0W
```

```
row [28] M=2  T=283.1K  ID=500.0  RL=79.10  actual_power=679.9W  predicted_p
ower=261.8W
row [29] M=0  T=283.1K  ID=700.0  RL=4.80  actual_power=249.8W  predicted_po
wer=249.2W
row [30] M=0  T=283.1K  ID=700.0  RL=29.70  actual_power=81.8W  predicted_po
wer=80.3W
row [31] M=1  T=283.1K  ID=700.0  RL=29.70  actual_power=327.4W  predicted_p
ower=327.6W
row [32] M=2  T=283.1K  ID=700.0  RL=29.70  actual_power=518.9W  predicted_p
ower=510.8W
row [33] M=0  T=283.1K  ID=700.0  RL=55.30  actual_power=43.9W  predicted_po
wer=51.8W
row [34] M=1  T=283.1K  ID=700.0  RL=55.30  actual_power=175.8W  predicted_p
ower=187.0W
row [35] M=2  T=283.1K  ID=700.0  RL=55.30  actual_power=703.3W  predicted_p
ower=364.1W
row [36] M=0  T=283.1K  ID=1000.0  RL=3.92  actual_power=658.9W  predicted_p
ower=284.6W
row [37] M=1  T=283.1K  ID=1000.0  RL=3.92  actual_power=272.9W  predicted_p
ower=273.4W
row [38] M=2  T=283.1K  ID=1000.0  RL=3.92  actual_power=136.4W  predicted_p
ower=160.7W
row [39] M=0  T=283.1K  ID=1000.0  RL=11.70  actual_power=220.7W  predicted_
power=220.9W
row [40] M=1  T=283.1K  ID=1000.0  RL=11.70  actual_power=883.1W  predicted_
power=868.5W
row [41] M=2  T=283.1K  ID=1000.0  RL=11.70  actual_power=407.2W  predicted_
power=821.0W
row [42] M=0  T=283.1K  ID=1000.0  RL=25.20  actual_power=102.5W  predicted_
power=99.4W
row [43] M=1  T=283.1K  ID=1000.0  RL=25.20  actual_power=410.0W  predicted_
power=410.5W
row [44] M=2  T=283.1K  ID=1000.0  RL=25.20  actual_power=877.2W  predicted_
power=878.4W
row [45] M=0  T=283.1K  ID=1000.0  RL=41.60  actual_power=62.0W  predicted_p
ower=62.1W
row [46] M=1  T=283.1K  ID=1000.0  RL=41.60  actual_power=248.3W  predicted_
power=234.6W
row [47] M=2  T=283.1K  ID=1000.0  RL=41.60  actual_power=993.5W  predicted_
power=510.7W
/tmp/ipykernel_1019621/2855066255.py:48: UserWarning: No artists with labels
found to put in legend.  Note that artists whose label start with an undersc
ore are ignored when legend() is called with no argument.
  plt.legend()
```




```
===== Validation Data Error Analysis =====
Mean Absolute Error (MAE): 202.0777 W
MAE as % of mean power: 64.99%
Root Mean Squared Error (RMSE): 259.9462 W
Max Absolute Error: 482.7832 W
Min Absolute Error: 0.4798 W
R-squared (R²): 0.2449
```

In [109…

```python
# Simplified version if you only have one Task 2.1 model available
input_conditions = [
    [10.0, 200, 50.0],
    [20.0, 200, 130.0],
    [10.0, 500, 40.0],
    [20.0, 500, 80.0],
    [20.0, 700, 30.0],
    [20.0, 700, 55.0],
    [10.0, 1000, 12.0],
    [20.0, 1000, 25.0],
    [20.0, 1000, 39.0]
]
```

```
results = []

for T_air_degC, I_D, R_L in input_conditions:
    T_air_K = T_air_degC + 273.15
    row_data = {'T_air': T_air_degC, 'I_D': I_D, 'R_L': R_L, 'powers': {}}

    # Predict power for M=0, 1, 2
    for mode in [0, 1, 2]:
        test_input = [[mode, T_air_K / Tamed, I_D / IDmed, R_L / RLmed]]
        test_array = np.array(test_input)
        outpt_power = model.predict(test_array, verbose=0)
        power_W = outpt_power[0][1] * Wdmed
        row_data['powers'][mode] = power_W

    # Predicted M_max,int is the mode with maximum power
    M_max_int = max(row_data['powers'], key=row_data['powers'].get)
    row_data['M_max_int_pred'] = M_max_int
    results.append(row_data)

# Display results
for idx, row in enumerate(results):
    print(f"Row {idx+1}: T={row['T_air']}°C, I_D={row['I_D']}, R_L={row['R_L
    print(f"  M=0: {row['powers'][0]:.2f}W | M=1: {row['powers'][1]:.2f}W |
    print(f"  → Predicted M_max,int = {row['M_max_int_pred']}")
```

```
Row 1: T=10.0°C, I_D=200, R_L=50.0
  M=0: 41.47W | M=1: 170.95W | M=2: 62.54W
  → Predicted M_max,int = 1
Row 2: T=20.0°C, I_D=200, R_L=130.0
  M=0: 10.99W | M=1: 63.40W | M=2: 168.56W
  → Predicted M_max,int = 2
Row 3: T=10.0°C, I_D=500, R_L=40.0
  M=0: 55.76W | M=1: 230.15W | M=2: 156.83W
  → Predicted M_max,int = 1
Row 4: T=20.0°C, I_D=500, R_L=80.0
  M=0: 86.59W | M=1: 170.05W | M=2: 258.15W
  → Predicted M_max,int = 2
Row 5: T=20.0°C, I_D=700, R_L=30.0
  M=0: 81.99W | M=1: 332.18W | M=2: 503.86W
  → Predicted M_max,int = 2
Row 6: T=20.0°C, I_D=700, R_L=55.0
  M=0: 52.83W | M=1: 190.36W | M=2: 361.60W
  → Predicted M_max,int = 2
Row 7: T=10.0°C, I_D=1000, R_L=12.0
  M=0: 213.81W | M=1: 871.57W | M=2: 831.02W
  → Predicted M_max,int = 1
Row 8: T=20.0°C, I_D=1000, R_L=25.0
  M=0: 103.46W | M=1: 435.25W | M=2: 881.90W
  → Predicted M_max,int = 2
Row 9: T=20.0°C, I_D=1000, R_L=39.0
  M=0: 67.27W | M=1: 251.11W | M=2: 574.53W
  → Predicted M_max,int = 2
```

In [ ]:

In [1]:
```
import math, numpy
#Part 2 input data: Mode number, Air temp (degC), ID (W/sqm), load resistanc
# - split into training and validation sets
xdata = [[0., 10.0, 200.0, 24.3],
 [1., 10.0, 200.0, 24.3],
 [2., 10.0, 200.0, 24.3],
 [0., 10.0, 200.0, 51.8],
 [1., 10.0, 200.0, 51.8],

 [2., 10.0, 200.0, 51.8],
 [0., 10.0, 200.0, 96.2],
 [1., 10.0, 200.0, 96.2],
 [2., 10.0, 200.0, 96.1],
 [0., 10.0, 200.0, 170.1],

 [1., 10.0, 200.0, 170.1],
 [2., 10.0, 200.0, 170.1],
 [0., 10.0, 500.0, 7.0],
 [1., 10.0, 500.0, 7.0],
 [2., 10.0, 500.0, 7.0],

 [0., 10.0, 500.0, 21.2],
 [1., 10.0, 500.0, 21.2],
 [2., 10.0, 500.0, 21.2],
 [0., 10.0, 500.0, 43.2],
 [1., 10.0, 500.0, 43.2],

 [1., 10.0, 700.0, 4.9],
 [2., 10.0, 700.0, 4.9],
 [0., 10.0, 700.0, 14.3],
 [1., 10.0, 700.0, 14.3],
 [2., 10.0, 700.0, 14.3],

 [2., 10.0, 500.0, 43.2],
 [0., 10.0, 500.0, 79.1],
 [1., 10.0, 500.0, 79.1],
 [2., 10.0, 500.0, 79.1],
 [0., 10.0, 700.0, 4.8],

 [0., 10.0, 700.0, 29.7],
 [1., 10.0, 700.0, 29.7],
 [2., 10.0, 700.0, 29.7],
 [0., 10.0, 700.0, 55.3],
 [1., 10.0, 700.0, 55.3],
 [2., 10.0, 700.0, 55.3],

 [0., 10.0, 1000.0, 3.92],
 [1., 10.0, 1000.0, 3.92],
 [2., 10.0, 1000.0, 3.92],
 [0., 10.0, 1000.0, 11.7],
 [1., 10.0, 1000.0, 11.7],

 [2., 10.0, 1000.0, 11.7],
 [0., 10.0, 1000.0, 25.2],
```

```
 [1., 10.0, 1000.0, 25.2],
 [2., 10.0, 1000.0, 25.2],
 [0., 10.0, 1000.0, 41.6],
 [1., 10.0, 1000.0, 41.6],
 [2., 10.0, 1000.0, 41.6],


#Part 2 output data for above specified Mode and conditons: VL (V) and Power
ydata = [[46.0, 87.3],
         [27.9, 64.2],
         [27.9, 32.1],
         [46.0, 40.9],
         [92.1, 163.9],
         [59.5, 68.5],
         [46.0, 22.0],
         [92.1, 88.2],
         [110.6, 127.2],
         [46.0, 12.4],
         [92.1, 49.9],
         [184.2, 199.6],
         [48.4, 335.3],
         [20.8, 124.3],
         [20.8, 62.1],
         [48.4, 110.7],
         [96.8, 442.9],
         [63.1, 188.2],
         [48.4, 54.3],
         [96.9, 217.3],
         [128.7, 383.6],
         [48.4, 29.6],
         [96.8, 118.7],
         [193.7, 474.8],
         [49.3, 496.1],
         [20.4, 171.2],
         [20.4, 85.6],
         [49.3, 169.9],
         [98.6, 679.9],
         [59.7, 249.8],
         [49.3, 81.8],
         [98.6, 327.4],
         [124.1, 518.9],
         [49.305, 43.9],
         [98.6, 175.8],
         [197.2, 703.3],
         [50.8, 658.9],
         [23.1, 272.9],
         [23.1, 136.4],
         [50.8, 220.7],
         [101.6, 883.1],
         [69.0, 407.2],
         [50.8, 102.5],
         [101.6, 410.0],
         [148.6, 877.2],
         [50.8, 62.0],
         [101.6, 248.3],
         [203.2, 993.5]]
```

```
xarray= numpy.array(xdata)
yarray= numpy.array(ydata)
print (xdata)
print (xarray)
print (ydata)
print (yarray)
```

```
  Cell In[1], line 4
    xdata = [[0., 10.0, 200.0, 24.3],
             ^
SyntaxError: '[' was never closed
```

In [ ]:

```
import math, numpy
#Part 1 input data: Air temp (degC), ID (W/sqm), load resistance (ohms)
# - split into training set and a randomly slected small validation set


#Part 1 output data: VL (V) and Power out (W)
ydata = [[18.9, 80.3],
 [23.5, 124.6],
 [24.8, 138.6],
 [25.6, 146.9],
 [19.2, 83.1],
 [25.0, 140.5],
 [26.5, 157.6],
 [27.3, 167.5],
 [19.4, 84.7],
 [26.4, 156.7],
 [28.1, 177.7],
 [29.0, 189.4],
 [22.4, 75.2],
 [24.8, 92.2],
 [25.8, 99.7],
 [26.4, 104.6],
 [23.6, 83.7],
 [26.5, 104.9],
 [27.6, 113.8],
 [28.3, 119.6],
 [24.8, 92.0],
 [28.1, 118.2],
 [29.3, 128.8],
 [30.1, 135.5],
 [23.5, 62.2],
 [25.4, 72.5],
 [26.3, 77.6],
 [26.9, 81.1],
 [25.0, 70.3],
 [27.1, 82.7],
 [28.1, 88.7],
 [28.7, 92.8],
 [26.5, 78.6],
 [28.8, 93.5],
 [29.9, 100.5],
 [30.6, 105.2]]
```

```
xarray= numpy.array(xdata)
yarray= numpy.array(ydata)
print (xdata)
print (xarray)
print (ydata)
print (yarray)
```

In [ ]:

```
'''>>>>> proto CodeP4.4F25
    V.P. Carey ME249, Fall 2025

Intro to Neural Network Modeling
Keras model for PV panel electrical power system'''

#import useful packages
import keras
import pandas as pd
from keras.models import Sequential
import numpy as np
import keras.backend as kb
import tensorflow as tf
#the following 2 lines are only needed for Mac OS machines
import os
os.environ['KMP_DUPLICATE_LIB_OK']='True'

#create input data array
# meadian values of input variables - DETERMINE APPROPRIATE MEDIAN VALUES AN
Tamed = 10.
IDmed = 500.
RLmed = 80.
#Part 2 input data: Air temp (degC), ID (W/sqm), load resistance (ohms)
xdata = []
xdata = [[10.0/Tamed, 200.0/IDmed, 24.3/RLmed],
 [10.0/Tamed, 200.0/IDmed, 51.8/RLmed],
 [10.0/Tamed, 200.0/IDmed, 96.2/RLmed],
 [10.0/Tamed, 200.0/IDmed, 133.2/RLmed],
 [10.0/Tamed, 200.0/IDmed, 170.1/RLmed],
 [10.0/Tamed, 500.0/IDmed, 7.0/RLmed],
 [10.0/Tamed, 500.0/IDmed, 21.2/RLmed],
 [10.0/Tamed, 500.0/IDmed, 43.2/RLmed],
 [10.0/Tamed, 500.0/IDmed, 61.2/RLmed],
 [10.0/Tamed, 500.0/IDmed, 79.1/RLmed],
 [10.0/Tamed, 1000.0/IDmed, 3.92/RLmed],
 [10.0/Tamed, 1000.0/IDmed, 11.7/RLmed],
 [10.0/Tamed, 1000.0/IDmed, 25.2/RLmed],
 [10.0/Tamed, 1000.0/IDmed, 33.4/RLmed],
 [10.0/Tamed, 1000.0/IDmed, 41.6/RLmed],
 [10.0/Tamed, 700.0/IDmed, 4.9/RLmed],
 [10.0/Tamed, 700.0/IDmed, 14.3/RLmed],
 [10.0/Tamed, 700.0/IDmed, 29.7/RLmed],
 [10.0/Tamed, 700.0/IDmed, 42.9/RLmed],
 [10.0/Tamed, 700.0/IDmed, 55.3/RLmed],
 [18.1/Tamed, 500.0/IDmed, 7.0/RLmed],
 [18.5/Tamed, 500.0/IDmed, 21.2/RLmed],
 [19.0/Tamed, 500.0/IDmed, 43.2/RLmed],
 [18.6/Tamed, 500.0/IDmed, 61.2/RLmed],
 [18.8/Tamed, 500.0/IDmed, 79.1/RLmed],
 [2.1/Tamed, 1000.0/IDmed, 3.92/RLmed],
 [2.0/Tamed, 1000.0/IDmed, 11.7/RLmed],
 [1.9/Tamed, 1000.0/IDmed, 25.2/RLmed],
 [2.3/Tamed, 1000.0/IDmed, 33.4/RLmed],
 [2.4/Tamed, 1000.0/IDmed, 41.6/RLmed],
```

```
 [0.5/Tamed, 700.0/IDmed, 4.9/RLmed],
 [0.7/Tamed, 700.0/IDmed, 14.3/RLmed],
 [1.0/Tamed, 700.0/IDmed, 29.7/RLmed],
 [0.8/Tamed, 700.0/IDmed, 42.9/RLmed],
 [0.2/Tamed, 700.0/IDmed, 55.3/RLmed],
 [15.2/Tamed, 200.0/IDmed, 24.3/RLmed],
 [15.4/Tamed, 200.0/IDmed, 51.8/RLmed],
 [21.2/Tamed, 1000.0/IDmed, 3.92/RLmed],
 [19.4/Tamed, 1000.0/IDmed, 11.7/RLmed],
 [19.2/Tamed, 1000.0/IDmed, 25.2/RLmed]]

xarray= np.array(xdata)
# print (xdata)
# print (xarray)

# values of output variable

#Part 2 output data: mode resulting in maximum power output Mmax
ydata = []
ydata = [[0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
```

```
 [0.],
 [1.],
 [2.] ]
yarray= np.array(ydata)
# print (ydata)
# print (yarray)

# Change the labels above from categorical to one-hot encoding
ydataCatOHE = []
#classes = [0 1 2]
# Change the labels above from categorical to one-hot encoding
ydataCatOHE = [
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
]


ydataCatOHEarray= np.array(ydataCatOHE)
```

```
# print('ydataCatOHE = ', ydataCatOHE)
# print('ydataCatOHEarray = ', ydataCatOHEarray)


#  ADD CODE TO PARTITION THE DATA HERE - MANUALLY OR WITH THE CODE IN THE PR

#make sure scikit-learn (vers 1.0.2 or later) is installed
#in your python 3.7 environment
from sklearn.model_selection import train_test_split
train_X,valid_X,train_label,valid_label = train_test_split(xarray,
ydataCatOHEarray, test_size=0.25, random_state=13)
# print to check the shape of training and validation set
print(train_X.shape,valid_X.shape,train_label.shape,valid_label.shape)

#you can add these lines of code at the end to see how the labels are #divid
# between the sets:
# print("training OHE labels:")
# print(train_label)
# print("validation OHE labels:")
# print(valid_label)

num_classes = 3
```

In [ ]:
```
# define neural network model
#In the first layer, the input shape is provided, which is 3 in our case.
#As seen below, we have included three dense layers.
#The activation function we have chosen is elu, which stands for exponential

from keras import backend as K
#initialize weights with values between -0.2 and 0.5
initializer = keras.initializers.RandomUniform(minval= -0.2, maxval=0.5)

model = keras.Sequential([
    keras.layers.Dense(13, activation='elu', input_shape=[3],  kernel_initia
    keras.layers.Dense(26, activation='elu',  kernel_initializer=initializer
    keras.layers.Dense(13, activation='elu', kernel_initializer=initializer)
    keras.layers.Dense(num_classes, activation='softmax')
  ])
#ADD OUTPUT LAYER, REMOVE DROPOUTS FOR FIRST PART, ADD LATER
```

In [ ]:
```
#COMPLIE THE MODEL
'''After the model is created, you compile it using the Adam optimizer, one
most popular optimization algorithms.

Additionally, you specify the loss type which is categorical cross entropy w
is used for multi-class classification.  Lastly, you specify the metrics as
analyze while the model is training.'''
#COMPLETE THE model.compile STATEMENT BELOW
model.compile(loss=keras.losses.categorical_crossentropy,
optimizer=keras.optimizers.Adam(), metrics=['accuracy'])


#SUMMARIZE MODEL
'''To visualize the layers that created in the above step, use the summary f
```

```
This will show some parameters (weights and biases) in each layer and also t
model.summary()
```

In [ ]:
```
#TRAIN THE MODEL


# Add an early stopping callback
es = keras.callbacks.EarlyStopping(
    monitor='loss',
    mode='min',
    patience = 2000,
    restore_best_weights = True,
    verbose=1)
# Add a checkpoint where loss is minimum, and save that model
mc = keras.callbacks.ModelCheckpoint('best_model.keras', monitor='loss',
                     mode='min',  verbose=1, save_best_only=True)

#COMPLETE THE model.fit STATEMENT BELOW
model_train = model.fit(train_X, train_label, epochs=2000,verbose=1,validati
loss_hist = model_train.history['loss']
#The above line will return a dictionary, access it's info like this:
best_epoch = np.argmin(model_train.history['loss']) + 1
print ('best epoch = ', best_epoch)
print('smallest loss =', np.min(loss_hist))
```

In [ ]:
```
test = [] #specifies a test input data set
outpt=[]  #output of model for test input

#first point (data row [0]) comparison of data and prediction

#======================
#example code to convert one hot output to predicted Mmax integer
test = [[ xarray[0][0] , xarray[0][1] , xarray[0][2] ]]
testarray = np.array(test)
outpt = model.predict(testarray)
Mmaxint = np.argmax(np.round(outpt[0]))  # np.argmax returns the index of th
#first input data row:  normalized Air temp (degC), ID (W/sqm), load resista
print ('row [0] data:  Ta= ', xarray[0][0]*Tamed, ', ID= ', xarray[0][1]*IDm
    ' RL= ', xarray[0][2]*RLmed)
print (' pred Mmax= ', outpt[0],' Mmaxint = ', Mmaxint,' data Mmax= ', ydata

print (' ')
#Predict Mmaxint for arbitray specified input data:
print ('arbitrary input data:  Ta= ', 10., ', ID= ', 1000., ' RL= ', 41.6)
test = [[ 10.0/Tamed, 1000.0/IDmed, 41.6/RLmed ]]
testarray = np.array(test)
outpt = model.predict(testarray)
Mmaxint = np.argmax(np.round(outpt[0]))
print (' pred Mmax= ', outpt[0],' Mmaxint = ', Mmaxint,)

#======================
print (' ')
print('training versus validation comparisons')

#Model Evaluation on the Training Set
```

```
train_eval = model.evaluate(train_X, train_label, verbose=1)  #changed to ve
print('train loss:', train_eval[0])
print('train accuracy:', train_eval[1])

#Model Evaluation on the Validation Set
test_eval = model.evaluate(valid_X, valid_label, verbose=1)  #changed to ver
print('validation loss:', test_eval[0])
print('validation accuracy:', test_eval[1])
```

In [ ]:

In [ ]:

In [ ]:

```
'''>>>>> proto CodeP4.4F25
    V.P. Carey ME249, Fall 2025

Intro to Neural Network Modeling
Keras model for PV panel electrical power system'''

#import useful packages
import keras
import pandas as pd
from keras.models import Sequential
import numpy as np
import keras.backend as kb
import tensorflow as tf
#the following 2 lines are only needed for Mac OS machines
import os
os.environ['KMP_DUPLICATE_LIB_OK']='True'

#create input data array
# meadian values of input variables - DETERMINE APPROPRIATE MEDIAN VALUES AN
Tamed = 10.
IDmed = 500.
RLmed = 80.
#Part 2 input data: Air temp (degC), ID (W/sqm), load resistance (ohms)
xdata = []
xdata = [[10.0/Tamed, 200.0/IDmed, 24.3/RLmed],
 [10.0/Tamed, 200.0/IDmed, 51.8/RLmed],
 [10.0/Tamed, 200.0/IDmed, 96.2/RLmed],
 [10.0/Tamed, 200.0/IDmed, 133.2/RLmed],
 [10.0/Tamed, 200.0/IDmed, 170.1/RLmed],
 [10.0/Tamed, 500.0/IDmed, 7.0/RLmed],
 [10.0/Tamed, 500.0/IDmed, 21.2/RLmed],
 [10.0/Tamed, 500.0/IDmed, 43.2/RLmed],
 [10.0/Tamed, 500.0/IDmed, 61.2/RLmed],
 [10.0/Tamed, 500.0/IDmed, 79.1/RLmed],
 [10.0/Tamed, 1000.0/IDmed, 3.92/RLmed],
 [10.0/Tamed, 1000.0/IDmed, 11.7/RLmed],
 [10.0/Tamed, 1000.0/IDmed, 25.2/RLmed],
 [10.0/Tamed, 1000.0/IDmed, 33.4/RLmed],
 [10.0/Tamed, 1000.0/IDmed, 41.6/RLmed],
 [10.0/Tamed, 700.0/IDmed, 4.9/RLmed],
 [10.0/Tamed, 700.0/IDmed, 14.3/RLmed],
 [10.0/Tamed, 700.0/IDmed, 29.7/RLmed],
 [10.0/Tamed, 700.0/IDmed, 42.9/RLmed],
 [10.0/Tamed, 700.0/IDmed, 55.3/RLmed],
 [18.1/Tamed, 500.0/IDmed, 7.0/RLmed],
 [18.5/Tamed, 500.0/IDmed, 21.2/RLmed],
 [19.0/Tamed, 500.0/IDmed, 43.2/RLmed],
 [18.6/Tamed, 500.0/IDmed, 61.2/RLmed],
 [18.8/Tamed, 500.0/IDmed, 79.1/RLmed],
 [2.1/Tamed, 1000.0/IDmed, 3.92/RLmed],
 [2.0/Tamed, 1000.0/IDmed, 11.7/RLmed],
 [1.9/Tamed, 1000.0/IDmed, 25.2/RLmed],
 [2.3/Tamed, 1000.0/IDmed, 33.4/RLmed],
 [2.4/Tamed, 1000.0/IDmed, 41.6/RLmed],
```

```
 [0.5/Tamed, 700.0/IDmed, 4.9/RLmed],
 [0.7/Tamed, 700.0/IDmed, 14.3/RLmed],
 [1.0/Tamed, 700.0/IDmed, 29.7/RLmed],
 [0.8/Tamed, 700.0/IDmed, 42.9/RLmed],
 [0.2/Tamed, 700.0/IDmed, 55.3/RLmed],
 [15.2/Tamed, 200.0/IDmed, 24.3/RLmed],
 [15.4/Tamed, 200.0/IDmed, 51.8/RLmed],
 [21.2/Tamed, 1000.0/IDmed, 3.92/RLmed],
 [19.4/Tamed, 1000.0/IDmed, 11.7/RLmed],
 [19.2/Tamed, 1000.0/IDmed, 25.2/RLmed]]

xarray= np.array(xdata)
print (xdata)
print (xarray)

# values of output variable

#Part 2 output data: mode resulting in maximum power output Mmax
ydata = []
ydata = [[0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
 [2.],
 [2.],
 [2.],
 [0.],
 [1.],
```

```
 [0.],
 [1.],
 [2.] ]
yarray= np.array(ydata)
print (ydata)
print (yarray)

# Change the labels above from categorical to one-hot encoding
ydataCatOHE = []
#classes = [0 1 2]
# Change the labels above from categorical to one-hot encoding
ydataCatOHE = [
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [0, 0, 1],  # 2
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [1, 0, 0],  # 0
 [0, 1, 0],  # 1
 [0, 0, 1],  # 2
]


ydataCatOHEarray= np.array(ydataCatOHE)
```

```
print('ydataCatOHE = ', ydataCatOHE)
print('ydataCatOHEarray = ', ydataCatOHEarray)


#  ADD CODE TO PARTITION THE DATA HERE - MANUALLY OR WITH THE CODE IN THE PR

#make sure scikit-learn (vers 1.0.2 or later) is installed
#in your python 3.7 environment
from sklearn.model_selection import train_test_split
train_X,valid_X,train_label,valid_label = train_test_split(xarray,
ydataCatOHEarray, test_size=0.25, random_state=13)
# print to check the shape of training and validation set
print(train_X.shape,valid_X.shape,train_label.shape,valid_label.shape)

#you can add these lines of code at the end to see how the labels are #divid
# between the sets:
print("training OHE labels:")
print(train_label)
print("validation OHE labels:")
print(valid_label)

num_classes = 3
```

In [ ]:
```
# define neural network model
#In the first layer, the input shape is provided, which is 3 in our case.
#As seen below, we have included three dense layers.
#The activation function we have chosen is elu, which stands for exponential

from keras import backend as K
#initialize weights with values between -0.2 and 0.5
initializer = keras.initializers.RandomUniform(minval= -0.2, maxval=0.5)

model = keras.Sequential([
keras.layers.Dense(13, activation='elu',
input_shape=[3],kernel_initializer=initializer),
keras.layers.Dense(26, activation='elu', kernel_initializer=initializer),
keras.layers.Dropout(0.25),
keras.layers.Dense(13, activation='elu', kernel_initializer=initializer),
keras.layers.Dropout(0.25),
keras.layers.Dense(num_classes, activation='softmax')

])
#ADD OUTPUT LAYER, REMOVE DROPOUTS FOR FIRST PART, ADD LATER
```

In [ ]:
```
#COMPLIE THE MODEL
'''After the model is created, you compile it using the Adam optimizer, one
most popular optimization algorithms.

Additionally, you specify the loss type which is categorical cross entropy w
is used for multi-class classification.  Lastly, you specify the metrics as
analyze while the model is training.'''
#COMPLETE THE model.compile STATEMENT BELOW
model.compile(loss=keras.losses.categorical_crossentropy,
optimizer=keras.optimizers.Adam(), metrics=['accuracy'])
```

```
#SUMMARIZE MODEL
'''To visualize the layers that created in the above step, use the summary f
This will show some parameters (weights and biases) in each layer and also t
model.summary()
```

In [ ]:
```
#TRAIN THE MODEL


# Add an early stopping callback
es = keras.callbacks.EarlyStopping(
    monitor='loss',
    mode='min',
    patience = 2000,
    restore_best_weights = True,
    verbose=1)
# Add a checkpoint where loss is minimum, and save that model
mc = keras.callbacks.ModelCheckpoint('best_model.keras', monitor='loss',
                     mode='min',  verbose=1, save_best_only=True)

#COMPLETE THE model.fit STATEMENT BELOW
model_train = model.fit(train_X, train_label, epochs=2000,verbose=1,validati
loss_hist = model_train.history['loss']
#The above line will return a dictionary, access it's info like this:
best_epoch = np.argmin(model_train.history['loss']) + 1
print ('best epoch = ', best_epoch)
print('smallest loss =', np.min(loss_hist))
```

In [ ]:
```
test = [] #specifies a test input data set
outpt=[]  #output of model for test input

#first point (data row [0]) comparison of data and prediction

#=====================
#example code to convert one hot output to predicted Mmax integer
test = [[ xarray[0][0] , xarray[0][1] , xarray[0][2] ]]
testarray = np.array(test)
outpt = model.predict(testarray)
Mmaxint = np.argmax(np.round(outpt[0]))  # np.argmax returns the index of th
#first input data row:  normalized Air temp (degC), ID (W/sqm), load resista
print ('row [0] data:  Ta= ', xarray[0][0]*Tamed, ', ID= ', xarray[0][1]*IDm
    ' RL= ', xarray[0][2]*RLmed)
print (' pred Mmax= ', outpt[0],' Mmaxint = ', Mmaxint,' data Mmax= ', ydata

print (' ')
#Predict Mmaxint for arbitray specified input data:
print ('arbitrary input data:  Ta= ', 10., ', ID= ', 1000., ' RL= ', 41.6)
test = [[ 10.0/Tamed, 1000.0/IDmed, 41.6/RLmed ]]
testarray = np.array(test)
outpt = model.predict(testarray)
Mmaxint = np.argmax(np.round(outpt[0]))
print (' pred Mmax= ', outpt[0],' Mmaxint = ', Mmaxint,)

#=====================
```

```
print (' ')
print('training versus validation comparisons')

#Model Evaluation on the Training Set
train_eval = model.evaluate(train_X, train_label, verbose=1)  #changed to ve
print('train loss:', train_eval[0])
print('train accuracy:', train_eval[1])

#Model Evaluation on the Validation Set
test_eval = model.evaluate(valid_X, valid_label, verbose=1)  #changed to ver
print('validation loss:', test_eval[0])
print('validation accuracy:', test_eval[1])
```

In [ ]:

```
import pandas as pd

# Your input data from the table
input_data = [
    [10.0, 200, 50.0],
    [20.0, 200, 130.0],
    [10.0, 500, 40.0],
    [20.0, 500, 80.0],
    [20.0, 700, 30.0],
    [20.0, 700, 55.0],
    [10.0, 1000, 12.0],
    [20.0, 1000, 25.0],
    [20.0, 1000, 39.0]
]

# Normalize and predict for all rows
results = []
for Tair, ID, RL in input_data:
    test = [[ Tair/Tamed, ID/IDmed, RL/RLmed ]]
    testarray = np.array(test)
    outpt = model.predict(testarray, verbose=0)
    Mmaxint = np.argmax(np.round(outpt[0]))
    results.append({
        'T_air': Tair,
        'I_D': ID,
        'R_L': RL,
        'Mmax_int_pred': Mmaxint
    })

# Display results
df_results = pd.DataFrame(results)
print(df_results)
```

In [ ]: